Continuous Co-Occurrence Query over Object Streams from Videos (long version)

Zhongshuai Zhang
Beijing Institute of Technology
zhang_zhongshuai@126.com

Jiadong Zhang
Northeastern Univ., China
jiadong@stumail.neu.edu.cn

Bin Wang
Northeastern Univ., China
binwang@mail.neu.edu.cn

Xiaochun Yang
Northeastern Univ., China
yangxc@mail.neu.edu.cn

***Abstract*—Object detection and multi-object tracking turn video feeds into object streams, in which each frame gives the set of tracked objects in view. A continuous co-occurrence query reports, at every frame, all groups of at least a given number of objects (the size threshold) that co-occur in at least a given number of frames (the duration threshold) of a sliding window. The state-of-the-art methods scan frame by frame and derive new states from stored ones; they cannot prune by the duration threshold and must maintain all maximal co-occurrence pairs of the window, which far outnumber the answers. We propose OWIM (Object-Wise Incremental Maintenance), which scans object by object instead. Since the frames in which a group co-occurs can only shrink as objects are added, OWIM discards a candidate as soon as it falls below the duration threshold or cannot reach the size threshold; we prove that this pruning remains safe as the window slides. At each frame, OWIM trims only the entries that contain the expiring frame and recomputes only the answers that contain the new frame, so its cost per frame does not depend on the number of maximal co-occurrence pairs in the window. On real video datasets, OWIM is 64 to 4,055 times faster than the state-of-the-art method under the default parameters and 200 to 2,342 times faster on long videos, with a 99th-percentile latency of at most 53 μs per frame.**

***Index Terms*—continuous queries, co-occurrence patterns, video analytics, sliding windows, incremental maintenance**

I. Introduction

Video data is rapidly increasing: large numbers of cameras are deployed in transportation systems, in cities, in shopping malls, and along roads. Queries that retrieve video content matching specific patterns, such as object co-occurrence patterns [1]–[3] and co-movement patterns [4], [5], play a crucial role in extracting value from videos. Such queries are made possible by advances in object detection and multi-object tracking, which make it practical to turn videos into structured data automatically: real-time detectors [6], [7] recognize the objects in every frame, and tracking-by-detection methods [8]–[10] assign each object an identifier that persists across frames. A video feed can thus be turned into a structured *object stream*, in which each frame gives the set of tracked objects in view.

The *continuous co-occurrence query* $CQ(w, d, k)$ [1] is a basic and common query over object streams, which asks *which objects in a video appear together, and for how long*. Over a sliding window of the most recent w frames, at every frame it reports all groups of at least k objects that appear together in at least d frames of the window. Each answer is a *maximal co-occurrence pair* (MCP), i.e., a group of objects together with all the frames in which they co-occur, where neither the group nor its frames can be extended. With $d < w$, an object may be missing from a few frames, so the query tolerates occlusions and missed detections. Take Figure 1 as an example. The objects in each frame of the video are detected and tracked, forming the object stream $O_0 = \{C\}$, $O_1 = \{A, B, D\}$, $O_2 = \{A, B, C\}$, $O_3 = \{A, B, C, D\}$, $O_4 = \{A, B, D\}$, and $O_5 = \{A, B, D\}$. The query $CQ(5, 3, 3)$ asks, in each window of $w = 5$ frames, for the MCPs in which at least three objects appear together in at least three frames. At $t = 4$, the window W_4 (frames 0–4) contains five MCPs: $(\{A, B\}, \{1, 2, 3, 4\})$, $(\{A, B, D\}, \{1, 3, 4\})$, $(\{C\}, \{0, 2, 3\})$, $(\{A, B, C\}, \{2, 3\})$, and $(\{A, B, C, D\}, \{3\})$. Only $(\{A, B, D\}, \{1, 3, 4\})$ is an answer, since its three objects appear together in three frames; $(\{A, B\}, \{1, 2, 3, 4\})$ and $(\{C\}, \{0, 2, 3\})$ have fewer than three objects, and $(\{A, B, C\}, \{2, 3\})$ and $(\{A, B, C, D\}, \{3\})$ have fewer than three frames. After frame 5 arrives and frame 0 expires, the answer becomes $(\{A, B, D\}, \{1, 3, 4, 5\})$.

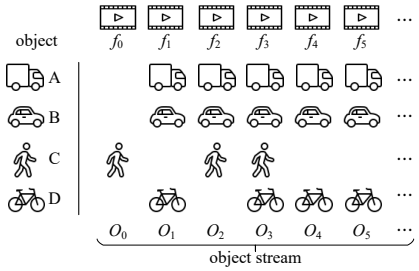


Fig. 1. A running example of the object stream.

Motivations and Challenges. Analyses of object groups are central to many applications: in traffic monitoring, groups of vehicles that travel together, such as platoons, are of interest for traffic management [4], [5]; in surveillance and public safety, groups of people who stay together in a scene need to be found [11]; and in curating data for autonomous driving, one needs to check whether the training videos contain enough scenes in which, e.g., several trucks appear together [1]. All of these applications require results to be produced continuously, frame by frame, over video streams. However, answering continuous co-occurrence queries efficiently faces three challenges. First, the number of MCPs in a window can grow exponentially with the number of objects in the window and

far exceeds the number of answers: W_4 in Figure 1 has five MCPs but one answer, and on real data the windows contain 290–6,882 MCPs per frame while there are only 0–4 answers per frame on average (Section V-B). Second, the results keep changing as the window slides at every frame: when a frame arrives, some groups gain a frame; when a frame expires, others lose one and may merge with other groups; and new groups emerge. Occlusions further scatter the frames of a group over the window, so that they cannot be summarized by a single interval. Third, query processing is under strict time constraints: since detection and tracking now run in real time [6]–[9], query maintenance must keep pace with the frame rate, leaving only a few tens of milliseconds per frame.

State-of-the-Art Efforts. Continuous co-occurrence queries have been studied in [1], which introduces a state-maintenance component with two methods, MFS and SSG, to maintain the co-occurrence states. Like an MCP, a state consists of an object set and a frame set, and the valid states are exactly the MCPs of the window. Both MFS and SSG scan frame by frame (*frame-wise*): the objects of a new frame are intersected with the object sets of existing states to find co-occurrences, and new states are derived from existing ones, inheriting their frames. The appearance history of a group of objects is therefore kept only in its state. A state with fewer than d frames may still reach d as new frames arrive, and discarding it would lose the frames accumulated so far; hence states cannot be pruned by d during maintenance. As a result, every MCP of the window is kept as a state regardless of d, and every new frame visits all existing states, so the cost grows with the total number of MCPs rather than with the size of the answer. In our experiments, SSG maintains 290–6,882 states per frame and takes up to 121 ms to process a single frame of a long video; on dataset D2, state maintenance alone exceeds the budget of about 33 ms per frame of a 30-fps video on 9% of the frames. To our knowledge, no existing method maintains the answers to continuous co-occurrence queries at a cost commensurate with the size of the answers.

Our Solution. We propose OWIM (Object-Wise Incremental Maintenance), whose main idea is to scan *object by object* (*object-wise*) instead. The co-occurrence frame set of a group of objects is the intersection of the appearance frame sets of these objects, and it can only shrink as objects are added during the scan. Hence, a candidate (i.e., a group of objects generated during the scan) with fewer than d frames can be discarded as soon as it is generated, and a candidate that cannot reach k objects can also be discarded early; that is, OWIM prunes by both parameters d and k. Over a sliding window, both pruning rules are permanently safe: a stored answer that does not contain the new frame can only lose frames as the window slides, so once it has fewer than d frames it can never become an answer again; and every answer that contains the new frame can be recomputed from the objects of that frame, so a pruned co-occurrence group can be found again if its objects co-occur again. Therefore, the table $\mathcal{D}$ maintained by OWIM stores only the MCPs that satisfy both thresholds, and we prove that it is exactly the answer set at every frame.

OWIM records the frames in which each object appears in the window as an *appearance bitmap*. The co-occurrence frame set of a group is then computed by a bitwise AND of the bitmaps of its objects, and its size by a population count; since a machine word covers 64 frames, each such operation takes only $\lceil w/64 \rceil$ word operations. OWIM proceeds in three steps; we count their costs in word operations on appearance bitmaps. When the query starts, it runs an object-wise dynamic program once over the first window to obtain the initial answers, which takes $O(n \cdot |\mathcal{D}| \cdot \lceil w/64 \rceil)$ word operations, where n is the number of objects in the window and $|\mathcal{D}|$ is the largest size that $\mathcal{D}$ reaches during the dynamic program. At every frame, it first handles the expiring frame: with an index that groups entries by their oldest frame, it trims only the C_e entries that contain the expiring frame and merges the trimmed entries with the same frame set, which takes $O(C_e \cdot \lceil w/64 \rceil)$ word operations. It then handles the new frame: it reruns the dynamic program only on the objects O_t of the new frame, which yields all answers that contain the new frame, and replaces the old entries that they supersede; this takes $O(|O_t| \cdot |\mathcal{D}'| \cdot \lceil w/64 \rceil)$ word operations, where $|\mathcal{D}'|$ is the largest size of the working table of this run, which holds only candidates that contain the new frame. To support these steps, appearance bitmaps use a circular encoding, so that sliding the window changes only one bit of each affected bitmap and the keys of the stored entries remain valid. The cost per frame therefore depends only on the objects of the new and the expiring frames and on the affected entries, not on the total number of MCPs in the window.

Our contributions are as follows.

- We analyze the limitation of Frame-Wise Incremental Maintenance: it cannot prune by the duration threshold d during maintenance, resulting in it maintaining states whose size exceeds that of the answer by far (Section II).
- We propose an object-wise dynamic program for computing MCPs, together with two pruning rules by d and by k (Section III).
- We propose bject-Wise Incremental Maintenanc, and prove that both pruning rules remain safe as the window slides and that the table it maintains equals the answer set at every frame (Section IV).
- We evaluate OWIM on all datasets of [1]. Under the default parameters, OWIM is 64–4,055 times faster than SSG, the state-of-the-art method, and 200–2,342 times faster on long videos; its 99th-percentile latency per frame is at most 53 μs (Section V).

II. Preliminaries

A. Problem Definition

1) From Video Feeds to Object Streams: We consider a video feed $V = \langle f_0, f_1, f_2, \dots \rangle$, an unbounded sequence of frames captured by a camera. Off-the-shelf object detection and multi-object tracking models (e.g., YOLO [6] and DeepSORT [12]) are applied to each frame: the detector reports the objects, and the tracker assigns every object an identifier that persists across frames, including frames in which the object is

TABLE I
NOTATION

Symbol	Meaning
O_t	set of objects (identifiers) visible in frame f_t
$\mathcal{O}$	object stream $\langle O_0, O_1, O_2, \dots \rangle$
w	window size (number of frames)
W_t	window at time t: the last w frames
$F_t(o)$	appearance frame set (bitmap) of object o in W_t
$F_t(X)$	co-occurrence frame set of object set X in W_t
$cover_t(S)$	objects that appear in every frame of S
d	duration threshold: minimum number of co-occurring frames
k	size threshold: minimum number of objects
(X, S)	maximal co-occurrence pattern (MCP)
$\mathrm{MCP}(W_t)$	set of MCPs in the window W_t
Ans_t	answer set of $CQ(w, d, k)$ at time t
$\mathcal{D}$	table of entries $S \mapsto X$; equals Ans_t after frame t
$pos(i)$	position of frame i in the circular encoding: $i \bmod w$
$I[p]$	oldest-frame index: keys of $\mathcal{D}$ with oldest frame at p
W°	W_{t-1} without the expiring frame
$\mathcal{D}'$	working table of the arrival DP on the new frame

temporarily occluded. Improving the accuracy of these models is orthogonal to this paper; we treat their output as given.

Let $\mathcal{ID}$ be the set of object identifiers. After detection and tracking, frame f_t is represented by the set $O_t \subseteq \mathcal{ID}$ of objects visible in it. The input to our problem is therefore the *object stream* $\mathcal{O} = \langle O_0, O_1, O_2, \dots \rangle$, which arrives one frame at a time. For example, Figure 1 shows an object stream. Four objects A, B, C, D appear in frames f_0–f_5: $O_0 = \{C\}$, $O_1 = \{A, B, D\}$, $O_2 = \{A, B, C\}$, $O_3 = \{A, B, C, D\}$, $O_4 = \{A, B, D\}$, and $O_5 = \{A, B, D\}$.

2) Co-occurrence in a Sliding Window: Queries are evaluated over a sliding window of the most recent w frames. At time t, the window is $W_t = \{\max(0, t - w + 1), \dots, t\}$; it advances by one frame whenever a new frame arrives.

For an object o, its *appearance frame set* in the window is

$$F_t(o) = \{\, i \in W_t \mid o \in O_i \,\}. \tag{1}$$

For a non-empty object set $X \subseteq \mathcal{ID}$, the frames in which all objects of X appear together form its *co-occurrence frame set*

$$F_t(X) = \bigcap\nolimits_{o \in X} F_t(o). \tag{2}$$

Occlusions are common in video, e.g., a car may be hidden behind a truck for a few frames and then reappear with the same identifier. We therefore do not require the frames in $F_t(X)$ to be consecutive. Instead, a *duration threshold* d ($1 \le d \le w$) states that X must co-occur in at least d frames of the window, i.e., $|F_t(X)| \ge d$. Choosing $d < w$ tolerates up to $w - d$ frames in which some object of X is occluded or missed by the detector. For instance, with videos at 30 fps, $w = 300$ and $d = 240$ ask for objects that appear together for at least 8 seconds within a 10-second window.

3) Maximal Co-occurrence Patterns: Co-occurrence is highly redundant. If X co-occurs in the frames S, every non-empty subset of X co-occurs in (at least) S, and X also co-occurs in every subset of S. Reporting all such combinations would overwhelm users with answers that convey the same information. We therefore report each group of objects together with all the frames in which it co-occurs, and each set of frames together with all the objects that co-occur in it.

Given a frame set $S \subseteq W_t$, let

$$cover_t(S) = \{\, o \in \mathcal{ID} \mid S \subseteq F_t(o) \,\} \tag{3}$$

be the objects that appear in every frame of S.

Definition 1 (Maximal co-occurrence pattern)**.** *Given the window W_t, a pair (X, S) with $\emptyset \ne X \subseteq \mathcal{ID}$ and $\emptyset \ne S \subseteq W_t$ is a* maximal co-occurrence pattern *(MCP) if and only if*

1) *$S = F_t(X)$, i.e., S contains every frame of the window in which all objects of X appear; and*
2) *$X = cover_t(S)$, i.e., no other object appears in all frames of S.*

Condition 1 says that S is the full co-occurrence frame set of X. Condition 2 additionally requires X to contain every object that appears in all frames of S. As a consequence, every frame set and every object set belongs to at most one MCP.

For example in Figure 1, let $w = 5$ and $t = 4$, so $W_4 = \{0, 1, 2, 3, 4\}$. The window contains five MCPs: $(\{A, B\}, \{1, 2, 3, 4\})$, $(\{A, B, D\}, \{1, 3, 4\})$, $(\{C\}, \{0, 2, 3\})$, $(\{A, B, C\}, \{2, 3\})$, and $(\{A, B, C, D\}, \{3\})$. The pair $(\{A, D\}, \{1, 3, 4\})$ is not an MCP: B also appears in frames 1, 3, and 4. Likewise, $(\{A, B\}, \{1, 3, 4\})$ is not an MCP, since A and B also co-occur in frame 2.

4) Continuous Co-occurrence Queries: We define the continuous co-occurrence query over an object stream using a sliding-window model.

Definition 2 (Continuous co-occurrence query)**.** *Let $\mathrm{MCP}(W_t)$ denote the set of MCPs in the window W_t. Given an object stream $\mathcal{O}$, a window size w, a duration threshold d ($1 \le d \le w$), and a size threshold $k \ge 1$, the* continuous co-occurrence query $CQ(w, d, k)$ *reports, at every time t, the answer set*

$$Ans_t = \{\, (X, S) \in \mathrm{MCP}(W_t) \mid |S| \ge d,\ |X| \ge k \,\}. \tag{4}$$

That is, at every frame we report every MCP of at least k objects that has co-occurred in at least d of the last w frames.

Problem statement. Given $CQ(w, d, k)$ over an object stream, maintain Ans_t as each new frame O_t arrives, with as little processing per frame as possible. Consecutive windows W_{t-1} and W_t share $w - 1$ frames, so Ans_{t-1} and Ans_t largely overlap; the challenge is to exploit this overlap instead of recomputing the answers of every window from scratch.

Comsidering $CQ(5, 3, 3)$ at $t = 4$ in Figure 1, the answer set Ans_4 contains a single MCP, $(\{A, B, D\}, \{1, 3, 4\})$. Another MCP $(\{A, B\}, \{1, 2, 3, 4\})$ lasts long enough ($4 \ge 3$ frames) but has only two objects. When f_5 arrives, frame 0 leaves the window, and frame 5 enters. The answer set Ans_5 again contains a single MCP, $(\{A, B, D\}, \{1, 3, 4, 5\})$: the same object set, now with one more frame. Meanwhile, $(\{C\}, \{0, 2, 3\})$ disappears from the set of MCPs, because the frames that remain for C, namely $\{2, 3\}$, are shared with A and B and are now covered by $(\{A, B, C\}, \{2, 3\})$.

B. Recap of Frame-Wise Incremental Maintenance

Previous work evaluated co-occurrence queries over video feeds in [1]. It introduces a state-maintenance component with two methods, Marked Frame Set (MFS) and Sparse State Graph (SSG), to maintain the co-occurrence states. A state pairs an object set with the frames of the window in which all of its objects appear; one state is kept for each distinct object set, and the valid states are exactly the MCPs of the window (Definition 1). States are maintained frame by frame: when a new frame arrives, its object set is intersected with the object set of every existing state, and each intersection either appends the new frame to an existing state or creates a new one; meanwhile, the expiring frame is removed from all states. This design offers three key advantages. First, states are reused across windows, so processing a new frame requires only one intersection per state instead of recomputing the whole window. Second, the MFS technique marks selected frames such that a state is valid if and only if at least one of its frames is marked; states that are no longer maximal can thus be detected without comparing frame sets. Third, the SSG organizes the states into a forest in which the object set of each child is contained in that of its parent; once a state has an empty intersection with the new frame, the intersections for all of its descendants are skipped.

Frame-Wise Incremental Maintenance has to maintain states whose size is far beyond that of the answer. By Definition 1, an MCP is maximal in two respects: in its frame set and in its object set. In such frame-wise incremental maintenance, new states are derived from existing ones: the object set of a new state is the intersection of the new frame with a stored object set, and its frames are inherited from the stored state. So the object set is maximal by construction, whereas the maximality of the frame set must be maintained separately. The frame set of a state can grow during maintenance: its objects co-occur again in a new frame. So a state whose objects have co-occurred in fewer than d frames may still reach d as the window slides, and discarding it would lose the frames accumulated so far. Hence, the duration condition $|S| \geq d$ can be checked only when answers are reported, and cannot be used to prune states during maintenance. As a result, every MCP of the window has to be maintained regardless of d. The per-frame cost thus grows with the total number of MCPs in the window, which can exceed the size of the answer set by orders of magnitude.

To reduce this cost, we propose Object-Wise Incremental Maintenance. It obtains the co-occurrence frame set by intersecting the appearance frame sets of objects, so the co-occurrence frame set shrinks as more objects are added, and the frame set is maximal by construction, whereas the maximality of the object set needs to be maintained separately. This enables pruning by the duration threshold during maintenance: since every answer satisfies $|S| \geq d$, a candidate with fewer than d frames is discarded as soon as it is generated. Moreover, further pruning is available: since every answer satisfies $|X| \geq k$, a candidate that cannot reach k objects even with all remaining objects is discarded early during the object-wise scanning. Over a sliding window, both pruning rules, by d and k, are permanently safe. This is in sharp contrast to Frame-Wise Incremental Maintenance, where every frame visits all MCPs of the window; in Object-Wise Incremental Maintenance, the larger d is, the fewer MCPs are kept and processed.

object	f_0	f_1	f_2	f_3	f_4	f_5
A	0	1	1	1	1	1
B	0	1	1	1	1	1
C	1	0	1	1	0	0
D	0	1	0	1	1	1

Fig. 2. The appearance bitmaps corresponding to the example in Fig. 1.

III. Object-wise Computation of MCPs

This section shows how to compute the answer set Ans_t of a single, fixed window W_t by scanning objects rather than frames. We introduce the appearance bitmaps that record where objects appear, together with their operations (Section III-A), the object-wise dynamic program that produces all MCPs of the window (Section III-B), and the pruning rules for the duration threshold d and the size threshold k (Section III-C). Finally, we combine them into a complete algorithm and describe its implementation (Section III-D). Since t is fixed throughout this section, we write $F(o)$, $F(X)$, and $cover(S)$ for $F_t(o)$, $F_t(X)$, and $cover_t(S)$.

A. Appearance Bitmaps and the Object-wise Scan

Let $o_1, \ldots, o_n$ be the objects that appear in W_t. Each object o records its appearance frame set $F(o)$ in a w-bit bitmap, called its *appearance bitmap*: every frame of the window corresponds to one bit, and the j-th bit is 1 if and only if o appears in the j-th frame of the window.

Two basic operations work on appearance bitmaps: the bitwise AND, and the population count, which counts the 1-bits of a bitmap. Both work on machine words; a bitmap occupies $\lceil w/64 \rceil$ words, so each operation takes $\lceil w/64 \rceil$ word operations. Since $F(X) = \bigcap_{o \in X} F(o)$ by definition, ANDing the appearance bitmaps of the objects in a object set X yields the bitmap of its co-occurrence frame set $F(X)$, and a population count of the result yields the number $|F(X)|$ of frames in which the object set co-occurs. In general, every frame set $S \subseteq W_t$ is represented by a w-bit bitmap in the same way, so that $S \cap S'$ is a bitwise AND and $|S|$ is a population count. We use a frame set and its bitmap interchangeably.

Fig. 2 shows the appearance bitmaps corresponding to the example in Fig. 1. Consider the object stream with $w = 5$ and $t = 4$, so $W_4 = \{0, 1, 2, 3, 4\}$. Each bitmap has five bits, listing frames 0–4 from left to right, and fits in a single machine word. Object A appears in frames 1–4, so $F(A) = \texttt{01111}$; likewise, $F(B) = \texttt{01111}$, $F(C) = \texttt{10110}$ (frames 0, 2, and 3), and $F(D) = \texttt{01011}$ (frames 1, 3, and 4). The co-occurrence frame set of the object set $\{A, B, D\}$ takes two bitwise ANDs, $\texttt{01111} \wedge \texttt{01111} \wedge \texttt{01011} = \texttt{01011}$, i.e., frames $\{1, 3, 4\}$,

and a population count of the result gives $|F(\{A, B, D\})| = 3$: the three objects co-occur in three frames.

We compute MCPs by object-wise scanning: we add objects one at a time, and each added object o costs one bitwise AND of the current co-occurrence frame set with $F(o)$, i.e., $F(X \cup \{o\}) = F(X) \cap F(o)$. Two simple properties make this scan order attractive.

Lemma 1 (Monotonicity)**.** *For any object sets $\emptyset \neq X \subseteq Y$, we have $F(Y) \subseteq F(X)$; that is, adding objects to an object set never adds frames to its co-occurrence frame set.*

Lemma 1 follows directly from the properties of intersection: ANDing with $F(o)$ can turn a 1 into a 0 but never a 0 into a 1, so we omit the proof. It shows that frame sets only shrink during the object-wise scan, which is what makes pruning by d possible (Section III-C). Two questions remain: first, whether every frame set obtained by intersecting objects corresponds to an MCP of the window; and second, which object set to keep when several object sets yield the same frame set. Lemma 2 answers both.

Lemma 2 (Frame sets of MCPs)**.** *For every non-empty object set Y with $F(Y) \neq \emptyset$, the pair $(cover(F(Y)), F(Y))$ is an MCP. Moreover, if some object set has exactly S as its co-occurrence frame set, then $cover(S)$ is the largest such set.*

Proof. Let $S = F(Y)$ and $X = cover(S)$. Since every object of Y appears in all frames of S, we have $Y \subseteq X$ and hence $F(X) \subseteq F(Y) = S$ by Lemma 1; since every object of X appears in all frames of S, we also have $S \subseteq F(X)$. Thus $S = F(X)$ and $X = cover(S)$, which are the two conditions of Definition 1. For the second claim, every object set Y' with $F(Y') = S$ satisfies $Y' \subseteq cover(S)$, and $F(cover(S)) = S$ by the first claim applied to Y'. □

The first claim of Lemma 2 answers the first question: every non-empty frame set $F(Y)$ produced by the object-wise scan is the frame set of some MCP, and the scan never produces a frame set that corresponds to no MCP. In other words, the object-wise scan satisfies condition 1 of Definition 1 by construction. The second claim answers the second question: for each frame set, it suffices to keep the largest object set of objects that produces it. The object set kept is $cover(S)$, which satisfies condition 2 without computing $cover(S)$ separately.

For example, consider again the window W_4 of Figure 2. For Lemma 1, start from $\{A\}$ and add D and then C: the co-occurrence frame set shrinks from $\{1, 2, 3, 4\}$ (`01111`) to $\{1, 3, 4\}$ (`01011`) and then to $\{3\}$ (`00010`).

For the first claim of Lemma 2, take $Y = \{A, D\}$, so $F(Y) = \{1, 3, 4\}$. The objects that appear in all of frames 1, 3, and 4 are A, B, and D (C is absent from frames 1 and 4), so $cover(\{1, 3, 4\}) = \{A, B, D\}$, which yields the MCP $(\{A, B, D\}, \{1, 3, 4\})$. Note that Y itself need not be the object set of an MCP: $(\{A, D\}, \{1, 3, 4\})$ is not an MCP because it misses B, but its frame set $\{1, 3, 4\}$ is the frame set of an MCP.

Processed	Table $\mathcal{D}_i$
none ($\mathcal{D}_0$)	empty
A ($\mathcal{D}_1$)	`01111` $\mapsto \{A\}$
A, B ($\mathcal{D}_2$)	`01111` $\mapsto \{A, B\}$
A, B, C ($\mathcal{D}_3$)	`01111` $\mapsto \{A, B\}$, `10110` $\mapsto \{C\}$, `00110` $\mapsto \{A, B, C\}$
A, B, C, D ($\mathcal{D}_4$)	`01111` $\mapsto \{A, B\}$, `10110` $\mapsto \{C\}$, `00110` $\mapsto \{A, B, C\}$, `01011` $\mapsto \{A, B, D\}$, `00010` $\mapsto \{A, B, C, D\}$

Fig. 3. The table $\mathcal{D}_i$ after each object of A, B, C, D in window W_4 is processed without pruning; bits list frames 0–4 from left to right.

For the second claim, the object sets whose co-occurrence frame set is exactly $\{1, 3, 4\}$ are $\{D\}$, $\{A, D\}$, $\{B, D\}$, and $\{A, B, D\}$. The largest is $\{A, B, D\} = cover(\{1, 3, 4\})$, and the other three are its subsets; hence, when these object sets yield the same frame set, it suffices to keep the largest one. The premise of the second claim cannot be dropped: no object set has exactly $\{1, 3\}$ as its co-occurrence frame set, since A, B, and D, the objects that appear in both frames 1 and 3, all appear in frame 4 as well. Here $cover(\{1, 3\}) = \{A, B, D\}$, whose co-occurrence frame set is $\{1, 3, 4\}$ rather than $\{1, 3\}$. The object-wise scan only produces frame sets of the form $F(Y)$, so this case never arises.

B. Object-wise MCP Computation by Dynamic Programming

We process the objects one at a time and maintain a table $\mathcal{D}$ that maps frame sets to object sets. After the first i objects have been processed, $\mathcal{D}_i[S]$ is the largest subset of $\{o_1, \ldots, o_i\}$ whose co-occurrence frame set is exactly S; it is undefined if no such subset exists, and $\mathcal{D}_0$ is empty. Only non-empty frame sets are stored, so a candidate whose frame set becomes empty is discarded.

When object $o = o_{i+1}$ is processed, a subset either excludes o, in which case it is already recorded in $\mathcal{D}_i$, or includes o, in which case it extends some recorded object set with o and its frame set becomes $S' \cap F(o)$; the object set $\{o\}$ alone has frame set $F(o)$. Writing $f_i(S) = |\mathcal{D}_i[S]|$ (and 0 if $\mathcal{D}_i[S]$ is undefined), and letting $P_i(S) = \{\, S' \in \mathrm{dom}(\mathcal{D}_i) \mid S' \cap F(o) = S \,\}$ be the keys of $\mathcal{D}_i$ that yield S when intersected with $F(o)$, the recurrence of the Dynamic Programming (DP) for every non-empty S is

$$f_{i+1}(S) = \max\{f_i(S), 1 + \max_{S' \in P_i(S)} f_i(S'), [S = F(o)]\}, \quad (5)$$

where $\mathrm{dom}(\mathcal{D}_i)$ is the set of keys of $\mathcal{D}_i$, the second term is present only if $P_i(S) \neq \emptyset$, and $[\cdot]$ is 1 if the condition holds and 0 otherwise. $\mathcal{D}_{i+1}[S]$ is the object set that attains the maximum, and $f_{i+1}(S) = 0$ means that $\mathcal{D}_{i+1}[S]$ is undefined.

The three terms of Equation (5) correspond to the three cases. The first term covers the object sets that exclude o and keeps the entries of $\mathcal{D}_i$; the second extends a recorded object set with o, which changes its frame set to $S' \cap F(o)$ and adds one object; and the third is the singleton object set $\{o\}$.

For example, consider again the window W_4 and process the objects in the order A, B, C, D, without pruning. Fig. 3 lists the table $\mathcal{D}_i$ that Equation (5) yields after each

object. When A is processed, $\mathcal{D}_0$ is empty and only the third term applies, which gives $\texttt{01111} \mapsto \{A\}$. When B is processed, all three terms fall on key $\texttt{01111}$: the first keeps $\{A\}$, the second extends $\{A\}$ with B to $\texttt{01111} \wedge \texttt{01111} = \texttt{01111}$, giving $\{A, B\}$, and the third gives $\{B\}$; the largest, $\{A, B\}$, is kept. When C is processed, the first term keeps $\texttt{01111} \mapsto \{A, B\}$, the second extends $\{A, B\}$ to $\texttt{01111} \wedge \texttt{10110} = \texttt{00110}$, giving $\{A, B, C\}$, and the third gives $\texttt{10110} \mapsto \{C\}$. When D is processed, the second term extends the three entries of $\mathcal{D}_3$ to $\texttt{01011} \mapsto \{A, B, D\}$, $\texttt{00010} \mapsto \{C, D\}$, and $\texttt{00010} \mapsto \{A, B, C, D\}$, and the third gives $\texttt{01011} \mapsto \{D\}$. For key $\texttt{01011}$, for instance, $f_4(\texttt{01011}) = \max\{\, f_3(\texttt{01011}),\ f_3(\texttt{01111}) + 1,\ 1\,\} = \max\{0,\ 3,\ 1\} = 3$, so $\mathcal{D}_4[\texttt{01011}] = \{A, B, D\}$. Likewise, $\{A, B, C, D\}$ wins over $\{C, D\}$ on key $\texttt{00010}$. The three entries of $\mathcal{D}_3$ are kept unchanged by the first term.

After all objects have been processed, $\mathcal{D}_4$ contains exactly all the five MCPs in the window W_4, as stated in Theorem 1.

Theorem 1. *After all n objects of the window W_t have been processed, $\mathcal{D}_n[S]$ is defined if and only if S is the frame set of an MCP of W_t, and in that case $(\mathcal{D}_n[S], S)$ is that MCP. Hence $\mathcal{D}_n$ contains exactly $\mathrm{MCP}(W_t)$, the set of all MCPs in W_t.*

Proof. We show by induction on i that $\mathcal{D}_i[S]$ is the largest subset of the first i objects whose frame set is exactly S. A largest such subset of the first $i+1$ objects either excludes o_{i+1}, or consists of o_{i+1} and a possibly empty set X' of the first i objects with $F(X') \cap F(o_{i+1}) = S$. Replacing a non-empty X' by $\mathcal{D}_i[F(X')]$ does not decrease its size, so Equation (5) attains it. Conversely, every term of Equation (5) is the size of a subset whose frame set is exactly S: $\mathcal{D}_i[S]$, $\mathcal{D}_i[S'] \cup \{o_{i+1}\}$ for $S' \in P_i(S)$, and $\{o_{i+1}\}$ if $S = F(o_{i+1})$. Hence $\mathcal{D}_n[S]$ is the largest object set whose frame set is exactly S, which is $\mathit{cover}(S)$ by Lemma 2. Every key S of $\mathcal{D}_n$ is of the form $F(Y)$ with $S \neq \emptyset$, so $(\mathcal{D}_n[S], S)$ is an MCP by Lemma 2; conversely, the frame set $S = F(X)$ of every MCP (X, S) is generated by the object set X itself. □

C. Pruning by d and k

By Theorem 1, without pruning $\mathcal{D}$ holds every MCP of the window, most of which are not answers. Two rules discard candidates as early as possible.

a) Rule 1 (duration): Objects with $|F(o)| < d$ do not take part in the scan at all; the remaining objects, in processing order, form the sequence L of length m. During the scan, a candidate whose frame set has fewer than d frames is discarded as soon as it is generated.

b) Rule 2 (size): Before the i-th object of L is processed, an entry (X, S) whose object set cannot reach k objects even if all $m - i + 1$ unprocessed objects joined it, i.e., $|X| + (m - i + 1) < k$, is removed.

Adding the two rules to the DP yields the *pruned DP*: only the objects in L are processed by Equation (5), one at a time; Rule 2 is applied before each object is processed; and a candidate produced by the second term of Equation (5) is discarded by Rule 1 if it has fewer than d frames. The singleton object set $\{o\}$ produced by the third term has $|F(o)| \geq d$ and needs no check. Lemma 3 shows that this pruning is safe.

Lemma 3. *Rules 1 and 2 never discard a candidate that leads to an answer: for every $(X, S) \in \mathit{Ans}_t$, once the pruned DP has processed all objects in L, the object set stored under key S is X.*

Proof. Let $(X, S) \in \mathit{Ans}_t$, so $|S| \geq d$ and $|X| \geq k$. Every object of X has $|F(o)| \geq |S| \geq d$ and thus belongs to L. For $i = 0, \ldots, m$, let $X_i = X \cap \{L[1], \ldots, L[i]\}$. We show by induction on i that, once the i-th object has been processed and $X_i \neq \emptyset$, $\mathcal{D}$ holds under key $F(X_i)$ a object set of at least $|X_i|$ objects. Consider $o = L[i+1]$. If $X_i \neq \emptyset$, the objects of $X \setminus X_i$ are among the $m - i$ unprocessed ones, so the object set stored under $F(X_i)$ has at least $|X_i| \geq |X| - (m - i) \geq k - (m - i)$ objects and survives Rule 2. If $o \notin X$, then $X_{i+1} = X_i$, and by the first term of Equation (5) this object set stays under key $F(X_i)$, possibly replaced by a larger one. If $o \in X$, then $F(X_{i+1}) = F(X_i) \cap F(o) \supseteq S$ by Lemma 1, so $|F(X_{i+1})| \geq d$; by the second term of Equation (5) (the third if $X_i = \emptyset$), key $F(X_{i+1})$ receives a object set of at least $|X_{i+1}|$ objects, which survives Rule 1. For $i = m$ we have $X_m = X$, so $\mathcal{D}[S]$ holds a object set of at least $|X|$ objects whose frame set is S; by Lemma 2, no such object set is larger than $\mathit{cover}(S) = X$, so $\mathcal{D}[S] = X$. □

Rule 2 deletes an entry from $\mathcal{D}$ outright, so later objects never extend it, and the same key may later be generated again from another entry. Lemma 3 shows that neither effect loses an answer.

Corollary 1. *Once the pruned DP has processed all objects in L, the entries (S, X) of $\mathcal{D}$ with $|X| \geq k$ form exactly Ans_t.*

Proof. By Lemma 3, every answer is in $\mathcal{D}$. Conversely, every entry (S, X) of $\mathcal{D}$ satisfies $F(X) = S$, and $|S| \geq d$ by Rule 1. If $|X| \geq k$, then $(\mathit{cover}(S), S)$ is an MCP by Lemma 2. Since $|\mathit{cover}(S)| \geq |X| \geq k$, this MCP is an answer, and Lemma 3 gives $X = \mathit{cover}(S)$. □

D. Algorithm and Implementation

Algorithm 1 combines the DP with the two pruning rules of into a complete algorithm. The table $\mathcal{D}$ is implemented as a hash table keyed by bitmaps, whose hash value is computed once per key over its $\lceil w/64 \rceil$ words. To process o, the algorithm first applies Rule 2 (line 5), takes a snapshot of the current entries (line 6), and inserts the singleton object set $\{o\}$ under key $F(o)$ (line 7, the third term of Equation (5)). Then, for every entry (S', X') of the snapshot, it computes $S' \cap F(o)$ by a bitwise AND and, if the result has at least d frames, inserts $X' \cup \{o\}$ under the resulting key (lines 8–11, the second term and Rule 1). Entries that are not replaced by a new insertion stay unchanged, which corresponds to the first term. An insertion keeps the larger of the new object set and the object set already stored under the same key (procedure INSERT), which is exactly the outer maximum of

Algorithm 1: Object-wise computation of MCPs for a single window

Input: appearance bitmaps $F(o)$ of the objects in W_t; thresholds d and k
Output: Ans_t

$L \leftarrow \langle o \mid |F(o)| \geq d \rangle$; $m \leftarrow |L|$ // Rule 1
$\mathcal{D} \leftarrow$ empty hash table keyed by bitmaps
for $i \leftarrow 1$ **to** m **do**
  $o \leftarrow L[i]$; $r \leftarrow m - i + 1$ // # unprocessed
  $\mathcal{D} \leftarrow \{ (S, X) \in \mathcal{D} \mid |X| + r \geq k \}$ // Rule 2
  $Q \leftarrow$ snapshot of the entries of $\mathcal{D}$
  INSERT$(\mathcal{D}, F(o), \{o\})$
  foreach $(S, X) \in Q$ **do**
    $S'' \leftarrow S \wedge F(o)$ // bitwise AND
    if $|S''| \geq d$ **then** // Rule 1
      INSERT$(\mathcal{D}, S'', X \cup \{o\})$
return $\{ (X, S) \mid (S, X) \in \mathcal{D},\ |X| \geq k \}$
Procedure INSERT$(\mathcal{D}, S, X)$
  if $\mathcal{D}[S]$ is undefined **or** $|\mathcal{D}[S]| < |X|$ **then**
    $\mathcal{D}[S] \leftarrow X$ // keep the larger object set

Object	$F(o)$	Table $\mathcal{D}$ after the step
A	01111	$01111 \mapsto \{A\}$
B	01111	$01111 \mapsto \{A, B\}$
C	10110	$01111 \mapsto \{A, B\}$, $10110 \mapsto \{C\}$; 00110 discarded by Rule 1
D	01011	$01111 \mapsto \{A, B\}$, $01011 \mapsto \{A, B, D\}$; 10110 removed by Rule 2

Fig. 4. Algorithm 1 on window W_4 with $d = k = 3$; bits list frames 0–4 from left to right.

Equation (5). Taking the snapshot first ensures that each object set is extended by o at most once. Hence, at the end of the i-th iteration, $\mathcal{D}$ is the table of the pruned DP after the first i objects, and by Corollary 1, Algorithm 1 returns exactly Ans_t.

We illustrate Algorithm 1 on the window W_4 of Fig. 2 with $d = k = 3$; Fig. 4 shows the run. All four objects appear in at least three frames and take part in the scan, i.e., $L = \langle A, B, C, D \rangle$ and $m = 4$. After A and B, the object set $\{A, B\}$ is stored under 01111. When C is processed, its singleton object set is stored under 10110, while $\{A, B, C\}$ yields 00110, which has only two frames and is discarded by Rule 1. Before D, the last object, is processed, the entry 10110 holds a single object and cannot reach three objects, so Rule 2 removes it. Processing D yields $01111 \wedge 01011 =$ 01011 for the object set $\{A, B, D\}$, which replaces the singleton $\{D\}$ under the same key. The algorithm returns $(\{A, B, D\}, \{1, 3, 4\})$, which is Ans_4. Without pruning, $\mathcal{D}$ would end as $\mathcal{D}_4$ in Fig. 3, with all five MCPs of the window; with pruning, only two of these entries remain, and only one of them is an answer.

IV. Incremental Maintenance over Sliding Windows

Consecutive windows share $w - 1$ frames, and their answer sets largely overlap. Incremental maintenance under the sliding window, rather than recalculating within each window, is urgent. This section maintains Ans_t incrementally. Section IV-A introduces the circular encoding for incremental maintenance, together with the oldest-frame index and the table $\mathcal{D}$ that work with it, and Section IV-B shows how MCPs change when a frame expires or arrives. Section IV-C presents the per-frame maintenance, proves its correctness and the safety of permanent pruning, and traces it on the running example; Section IV-D analyzes its cost.

A. Circular Encoding for Incremental Maintenance

The bitmaps of Section III encode positions relative to the window: when a frame arrives, every appearance bitmap shifts by one position and every key of $\mathcal{D}$ changes, so nothing computed for the previous window can be reused. To avoid shifting the appearance bitmaps as the window slides, and to keep the keys of stored entries valid across windows so that the entries can be reused, we extend the window-relative encoding of the appearance bitmaps to a *circular encoding* based on absolute positions.

Let $pos(i) = i \bmod w$ be the *position* of frame i; frame i corresponds to bit $pos(i)$ of a bitmap. The w frames of a window are consecutive, so they occupy w distinct positions; a frame keeps its position from its arrival until its expiration, and the new frame t takes over the position of the expiring frame $t - w$. Every frame set S of the window is encoded in the same way: bit $pos(i)$ is 1 for every $i \in S$. Hence a frame set that lies in both W_{t-1} and W_t has the same bitmap in both windows. For a fixed window, the circular encoding merely rotates the window-relative encoding by the same number of bits for all bitmaps, which preserves bitwise ANDs and population counts; the results of Section III and Algorithm 1 therefore apply unchanged.

a) Appearance bitmaps: For each object o that appears in the current window W_t, we store the circularly encoded bitmap of its appearance frame set $F_t(o)$, together with its number of frames $|F_t(o)|$, which is updated with every bit change.

b) Oldest-frame index: The *oldest-frame index* I stores, for each position p, the set $I[p]$ of the keys of $\mathcal{D}$ whose oldest frame is at position p; at initialization, every key of $\mathcal{D}$ is registered in I under its oldest frame. Since the expiring frame $t - w$ is the oldest frame of W_{t-1}, a frame set of W_{t-1} contains $t - w$ if and only if its oldest frame is $t - w$. Hence $I[pos(t)]$ lists exactly the entries affected by the expiration, and each entry is registered in a single list. The oldest frame of a key is found by scanning its bitmap from the position of the oldest frame of the window, which takes $\lceil w/64 \rceil$ word operations with a trailing-zero count instruction.

c) Table: The table $\mathcal{D}$ is the hash table of Section III, keyed by bitmaps, which maps the frame set S of each entry to its object set X. Unlike in Section III, $\mathcal{D}$ keeps only answers during incremental maintenance: after frame t has been processed, the entries of $\mathcal{D}$ are exactly the answers in Ans_t, so reporting the answers amounts to listing $\mathcal{D}$. Every entry is then an MCP of W_t; only while a frame is being

	position p	0	1	2	3	4
$t=4$	frame	0	1	2	3	4
	$F_4(A)$	0	1	1	1	1
	$F_4(B)$	0	1	1	1	1
	$F_4(C)$	1	0	1	1	0
	$F_4(D)$	0	1	0	1	1
$t=5$	frame	**5**	1	2	3	4
	$F_5(A)$	**1**	1	1	1	1
	$F_5(B)$	**1**	1	1	1	1
	$F_5(C)$	**0**	0	1	1	0
	$F_5(D)$	**1**	1	0	1	1

(a) Appearance bitmaps

p	$I[p]$, $t=4$	$I[p]$, $t=5$
0	`10110`	–
1	`01111` `01011`	`11111` `11011`
2	`00110`	`00110`
3	–	–
4	–	–

(b) Oldest-frame index I

Fig. 5. Circular encoding of the running example with $w = 5$. (a) Appearance bitmaps at $t = 4$ and $t = 5$; frame 5 takes over position 0 of the expired frame 0, and the bold column is the only one that changes. (b) The oldest-frame index I for $CQ(5, 2, 1)$ at $t = 4$ and $t = 5$.

processed can an entry temporarily fail to be one, e.g., before it is merged or replaced (Section IV-C).

Incremental maintenance processes each new frame t in two steps. If $t \geq w$, frame $t - w$ expires first: its bit is cleared from the bitmaps of its objects, bitmaps that become empty are recycled, and the entries that contain frame $t - w$ are trimmed. Then frame t arrives: its bit is set in the bitmaps of the objects of O_t, and the entries that contain frame t are computed by Algorithm 1 over the objects of O_t only. All other entries stay untouched.

For example, Fig. 5(a) shows the circular encoding of the object stream of Fig. 1 with $w = 5$. At $t = 4$, the frames of $W_4 = \{0, \ldots, 4\}$ occupy positions 0–4. When frame 5 arrives, frame 0 expires and frame 5 takes over its position 0, while frames 1–4 stay at positions 1–4, so only the column of bit 0 changes. A, B, and D appear in frame 5, so their bit 0 becomes 1, e.g., $F_5(D) = \texttt{11011}$, i.e., frames $\{1, 3, 4, 5\}$; C does not appear in frame 5, and $F_5(C) = \texttt{00110}$. Under the window-relative encoding, frames 1–5 of W_5 would occupy bits 0–4, so every bitmap would shift by one position; e.g., $F_5(D)$ would become `10111`. Under the circular encoding, the frame set $\{2, 3\}$ is `00110` in both windows, so the entry $\texttt{00110} \mapsto \{A, B, C\}$ stays valid after the window slides.

Fig. 5(b) illustrates the oldest-frame index for $CQ(5, 2, 1)$. At $t = 4$, $\mathcal{D}$ holds the four entries of $\mathcal{D}_4$ in Fig. 3 with at least two frames, each registered under its oldest frame: $I[0] = \{\texttt{10110}\}$, $I[1] = \{\texttt{01111}, \texttt{01011}\}$, and $I[2] = \{\texttt{00110}\}$. When frame 0 expires, only the entry in $I[0]$ is trimmed, and no other entry is checked. At $t = 5$, $\mathcal{D}$ holds `11111`, `11011`, and `00110` (Section IV-C). Bit 0 now stands for the newest frame 5, so the oldest frame of a key is found by scanning from $pos(1) = 1$, the position of the oldest frame of the window: `11111` and `11011` meet their first 1 at bit 1 and are registered in $I[1]$, while `00110` stays in $I[2]$. When frame 1 expires next, exactly these two entries are trimmed.

B. Properties of MCPs under Window Sliding

When frame t arrives, the window changes from W_{t-1} to W_t in two steps: if $t \geq w$, the oldest frame $e = t - w$ is removed, giving $W^\circ = W_{t-1} \setminus \{e\}$, and then the new frame t is added, giving $W_t = W^\circ \cup \{t\}$; if $t < w$, then $W^\circ = W_{t-1}$. To reason about the intermediate frame set W°, we extend the notation for windows to an arbitrary set W of frames. The *appearance frame set* of an object o in W is $F_W(o) = \{\, i \in W \mid o \in O_i \,\}$, i.e., the frames of W in which o appears. The *co-occurrence frame set* of a non-empty object set X in W is $F_W(X) = \bigcap_{o \in X} F_W(o)$, i.e., the frames of W in which all objects of X appear together. For a frame set $S \subseteq W$, $cover_W(S) = \{\, o \mid S \subseteq F_W(o) \,\}$ is the set of objects that appear in every frame of S. $\mathrm{MCP}(W)$ is the set of MCPs of W, defined by Definition 1 with W in place of W_t, and $Ans(W) = \{\, (X, S) \in \mathrm{MCP}(W) \mid |S| \geq d,\ |X| \geq k \,\}$ is the answer set over W. Thus $F_t = F_{W_t}$, $cover_t = cover_{W_t}$, and $Ans_t = Ans(W_t)$, and Lemmas 1 and 2 hold for any W with the same proofs.

This subsection answers the central question of incremental maintenance: given the answer set of W_{t-1}, i.e., $\mathcal{D}$, how to obtain the answer set of W_t? Since a slide of the window consists of an expiration and an arrival, we characterize how the set of MCPs changes in each step. These characterizations serve two purposes. First, they tell which entries each step must process and how. Second, they show that the processing needs only the stored answers and the appearance bitmaps, and no candidate that was pruned, which proves permanent pruning is safe. Lemma 4 is the basis, Lemma 5 characterizes the expiration, and Lemma 6 characterizes the arrival.

We first show with Lemma 4 that the object set of an entry does not change when other frames of the window change. Otherwise, even an entry that contains neither the expiring nor the new frame might need to be updated, and processing only the affected entries would be impossible.

Lemma 4 (Window-independent cover)**.** *For every non-empty frame set $S \subseteq W$, $cover_W(S) = \bigcap_{i \in S} O_i$. In particular, $cover_W(S)$ does not depend on W as long as W contains S; we therefore write $cover(S)$ for it.*

Proof. An object o belongs to $cover_W(S)$ if and only if $S \subseteq F_W(o)$, i.e., $o \in O_i$ for every $i \in S$. □

By Lemma 4, the object set of an entry depends only on its frame set. A slide of the window can therefore change an entry in only two ways: by removing the expiring frame from its frame set, or by adding the new frame to it.

Consider the expiration first. Once frame e is removed, the MCPs that contain e lose a frame, and several of them may end up with the same frame set; the MCPs that do not contain e keep their frame sets. Two questions arise: first, whether every MCP of W° can be obtained from an MCP of W, or whether new MCPs unrelated to those of W may appear; and second, which object set to keep when several MCPs have the same frame set after trimming. The two claims of Lemma 5 answer these questions in turn.

Lemma 5 (Expiration)**.** *Let W be a set of frames, $e \in W$, and $W^\circ = W \setminus \{e\}$.*

1) *For every MCP (X', S') of W°, the pair $(X', F_W(X'))$ is an MCP of W, and $F_W(X') \setminus \{e\} = S'$.*
2) *For every MCP (X, S) of W with $S \setminus \{e\} \neq \emptyset$, the frame set $S \setminus \{e\}$ is the frame set of an MCP of W°, whose object set $cover(S \setminus \{e\})$ is the largest object set among the MCPs of W whose frame sets become $S \setminus \{e\}$ when e is removed.*

Proof. (1) Let $S = F_W(X')$. Since $F_{W^\circ}(X') = F_W(X') \setminus \{e\}$ and $F_{W^\circ}(X') = S'$ by condition 1 of Definition 1, we have $S \setminus \{e\} = S'$. Every object of X' appears in all frames of S, so $X' \subseteq cover(S)$; conversely, $S' \subseteq S$ gives $cover(S) \subseteq cover(S') = X'$. Hence $cover(S) = X'$, and (X', S) satisfies both conditions of Definition 1 for W.

(2) Let $S' = S \setminus \{e\}$ and $X' = cover(S')$, so $X' \supseteq cover(S) = X$. By Lemma 1, $F_{W^\circ}(X') \subseteq F_{W^\circ}(X) = F_W(X) \setminus \{e\} = S'$, and $F_{W^\circ}(X') \supseteq S'$ by the definition of *cover*; hence (X', S') is an MCP of W°. Every MCP (X'', S'') of W with $S'' \setminus \{e\} = S'$ satisfies $X'' = cover(S'') \subseteq cover(S') = X'$, and by (1), $(X', F_W(X'))$ is itself such an MCP. □

In words, the MCPs of W° are obtained from those of W by removing e from every frame set and keeping, for every frame set that results more than once, the largest object set. Claim (1) also shows that every MCP of W° has an MCP in W with the same object set and at least as many frames. Hence every answer of W° comes from an answer of W: the expiration only needs to trim stored entries and never needs a pruned candidate. This is the key to proving permanent pruning safe.

Consider now the arrival. Adding the new frame t changes the set of MCPs in three ways. First, new MCPs whose frame sets contain t appear. Second, the existing MCPs whose objects all appear in frame t extend their frame sets by t, so the old pairs are no longer MCPs. Third, the remaining existing MCPs stay unchanged. The three claims of Lemma 6 characterize these three cases.

Lemma 6 (Arrival)**.** *Let W° be a set of frames, $t \notin W^\circ$ a new frame, and $W = W^\circ \cup \{t\}$.*

1) *The MCPs of W whose frame sets contain t are exactly the pairs $(cover(F_W(Y)), F_W(Y))$ for non-empty $Y \subseteq O_t$, and their object sets lie in O_t. Consequently, Algorithm 1 applied to the bitmaps $F_W(o)$ of the objects $o \in O_t$ returns exactly the answers in $Ans(W)$ whose frame sets contain t.*
2) *Let (X, S) be an MCP of W°. If $X \not\subseteq O_t$, then (X, S) is an MCP of W; otherwise, $(X, S \cup \{t\})$ is an MCP of W and (X, S) is not.*
3) *Every MCP of W whose frame set does not contain t is an MCP of W°.*

Proof. (1) If (X, S) is an MCP of W with $t \in S$, then $X = cover(S) \subseteq O_t$ by Lemma 4, and $S = F_W(X)$. Conversely, every non-empty $Y \subseteq O_t$ has $t \in F_W(Y)$, and $(cover(F_W(Y)), F_W(Y))$ is an MCP of W by Lemma 2. The proofs of Theorem 1, Lemma 3, and Corollary 1 therefore carry over when the scan is restricted to the objects of O_t: every frame set produced contains t, and the object set of every answer that contains t lies in O_t.

(2) If $X \not\subseteq O_t$, then $F_W(X) = F_{W^\circ}(X) = S$, and $cover(S) = X$ by Lemma 4. If $X \subseteq O_t$, then $F_W(X) = S \cup \{t\} \neq S$, so (X, S) violates condition 1 of Definition 1, whereas $cover(S \cup \{t\}) = cover(S) \cap O_t = X$.

(3) Let (X, S) be an MCP of W with $t \notin S$. Then $F_{W^\circ}(X) = F_W(X) \setminus \{t\} = S$ and $cover(S) = X$. □

Algorithm 2: Per-frame maintenance

Input: new frame t with objects O_t; window size w; thresholds d and k; bitmaps $F_{t-1}(o)$, table $\mathcal{D} = Ans_{t-1}$, and index I
Output: bitmaps $F_t(o)$, table $\mathcal{D} = Ans_t$, and index I

```
1  p ← t mod w
2  if t ≥ w then // expiration
3      foreach o ∈ O_{t−w} do
4          F_{W°}(o) ← F_{t−1}(o) with bit p cleared
5          if F_{W°}(o) is empty then remove it                    // recycle
6      foreach S ∈ I[p] do
7          X ← D[S]; remove S from D
8          S' ← S with bit p cleared
9          if |S'| ≥ d then INSERT(D, S', X)                       // Rule 1
10     I[p] ← ∅
11 foreach o ∈ O_t do // arrival
12     F_t(o) ← F_{W°}(o) with bit p set
13 N ← Algorithm 1 on {F_t(o) | o ∈ O_t}
14 foreach (X, S) ∈ N do
15     S⁻ ← S with bit p cleared
16     if D[S⁻] is defined and |D[S⁻]| = |X| then // replaced
17         remove S⁻ from D and I
18     INSERT(D, S, X)

19 Procedure INSERT(D, S, X)
20     if D[S] is undefined then
21         D[S] ← X; add S to I[pos(oldest frame of S)]
22     else if |D[S]| < |X| then D[S] ← X
```

Claim (1) shows that the MCPs that contain the new frame can be recomputed from the appearance frame sets of the objects in O_t alone, rather than derived from stored entries; this justifies running Algorithm 1 only on the objects of the new frame at each arrival. Claim (2) gives the condition under which a stored entry becomes invalid, which corresponds to the replacement test of Section IV-C. Claim (3) shows that no MCP without t newly appears, so no other entry needs to be checked. Together, Lemmas 5 and 6 provide everything that the per-frame maintenance needs, and Section IV-C builds Algorithm 2 on them.

C. Per-frame Maintenance

Algorithm 2 maintains $\mathcal{D}$ from Ans_{t-1} to Ans_t for every new frame $t > t_0$. Let $p = pos(t)$ be the position shared by the new frame t and the expiring frame $t - w$.

a) Expiration: If $t \geq w$, frame $e = t - w$ leaves the window (line 2). For each object of O_e, the algorithm clears bit p of its bitmap $F_{t-1}(o)$, which yields $F_{W^\circ}(o)$ (line 4); the bitmaps of the other objects are already 0 at this bit, i.e., $F_{W^\circ}(o) = F_{t-1}(o)$. In our implementation, O_e is obtained from a *frame ring* of length w that stores the object set

of each frame of the window: the object set of frame i is kept in slot $pos(i)$, and that of the new frame t overwrites the slot left by the expiring frame $t - w$. An object whose $F_{W^\circ}(o)$ is empty no longer appears in the window, and its bitmap is recycled (line 5), so only the objects of the current window keep appearance bitmaps. Next, the algorithm takes the entries that contain e from $I[p]$ and trims them (lines 6–9): each such entry is removed from $\mathcal{D}$, and bit p is cleared from its key S; if the trimmed frame set $S \setminus \{e\}$ still has at least d frames, the original object set X is put back into $\mathcal{D}$ with INSERT (Rule 1). The procedure INSERT of Algorithm 2 follows that of Algorithm 1: if the key is already in $\mathcal{D}$, it keeps the larger object set; its only additional step (line 21) registers a new key in I under its oldest frame. This merges the entries whose frame sets coincide once e is removed, as required by Lemma 5. Entries that do not contain e are not touched.

b) Arrival: Frame t then enters the window: bit p is set in the bitmap $F_{W^\circ}(o)$ of every object of O_t, which yields $F_t(o)$; for an object that newly appears, $F_{W^\circ}(o)$ is empty and a new bitmap is created (line 12); for the objects not in O_t, $F_t(o) = F_{W^\circ}(o)$. In our implementation, all these updates are made in place on a single bitmap per object, and O_t is also stored in slot p of the frame ring for the expiration of frame t. The entries that contain t are recomputed from the bitmaps rather than derived from stored entries: by Lemma 6(1), Algorithm 1 applied to the bitmaps $F_t(o)$ of the objects of O_t returns all answers whose frame sets contain t, with both pruning rules (line 13). This run uses its own working table, denoted $\mathcal{D}'$, which is separate from the maintained table $\mathcal{D}$. Before each answer (X, S) is inserted into $\mathcal{D}$ with INSERT (line 18), the *replacement test* checks the key $S^- = S \setminus \{t\}$ (line 16; an undefined $\mathcal{D}[S^-]$ fails the test). If $\mathcal{D}$ stores an object set under S^-, that set is $cover(S^-) \supseteq cover(S) = X$, so it has the same size as X if and only if it equals X, i.e., if and only if all of its objects appear in frame t. By Lemma 6(2), the old entry is then no longer an MCP and is replaced by the new entry with key S. After the expiration, no key of $\mathcal{D}$ contains bit p, so INSERT always inserts a new key here. Entries that contain neither e nor t are not touched.

c) Correctness: The following theorem shows that, after every frame, Algorithm 2 maintains exactly the answer set.

Theorem 2. *For every $t \geq t_0$, after Algorithm 2 has processed frame t, the table $\mathcal{D}$ equals Ans_t.*

Proof. By induction on t. After initialization, $\mathcal{D} = Ans_{t_0}$ by Corollary 1; if the query starts with the stream, both the window and $\mathcal{D}$ are empty. Assume $\mathcal{D} = Ans(W_{t-1})$ before frame t. We first show that $\mathcal{D} = Ans(W^\circ)$ after the expiration, which is immediate if $t < w$. Let $(X', S') \in Ans(W^\circ)$. By Lemma 5(1), (X', S) with $S = F_{W_{t-1}}(X')$ is an MCP of W_{t-1} with $|S| \geq |S'| \geq d$ and $|X'| \geq k$, so $\mathcal{D}[S] = X'$ before the expiration. Every other entry of $\mathcal{D}$ whose frame set becomes S' after trimming has a proper subset of X' as its object set, since distinct MCPs have distinct object sets and X' is the largest by Lemma 5(2). Hence $\mathcal{D}[S'] = X'$ after the expiration. Conversely, let $\mathcal{D}[S'] = X$ after the expiration.

Step	Table $\mathcal{D}$
initialization at $t_0 = 4$	$\texttt{01111} \mapsto \{A, B\}$, $\texttt{01011} \mapsto \{A, B, D\}$, $\texttt{10110} \mapsto \{C\}$, $\texttt{00110} \mapsto \{A, B, C\}$
expire frame 0: $F_{W^\circ}(C) = \texttt{00110}$	$\texttt{01111} \mapsto \{A, B\}$, $\texttt{01011} \mapsto \{A, B, D\}$, $\texttt{00110} \mapsto \{A, B, C\}$
arrive frame 5: $F_5(A) = F_5(B) = \texttt{11111}$, $F_5(D) = \texttt{11011}$	$\texttt{11111} \mapsto \{A, B\}$, $\texttt{11011} \mapsto \{A, B, D\}$, $\texttt{00110} \mapsto \{A, B, C\}$

Fig. 6. Algorithm 2 on the running example for $CQ(5, 2, 1)$: initialization at $t_0 = 4$, followed by frame 5. Bits list positions 0–4 from left to right; at $t = 5$, position 0 holds frame 5.

This entry results from an answer (X, S) of W_{t-1} with $S \setminus \{e\} = S'$ and $|S'| \geq d$. By Lemma 5(2), $(cover(S'), S')$ is an MCP of W° with $|cover(S')| \geq |X| \geq k$, i.e., an answer of W°, so $X = cover(S')$ by the first part.

Next, by Lemma 6(1), line 13 returns exactly the answers of W_t that contain t, and all of them are inserted into $\mathcal{D}$. Every answer of W_t that does not contain t is an answer of W° with the same object and frame sets by Lemma 6(3), so it is already in $\mathcal{D}$; its object set is not a subset of O_t, so the replacement test does not remove it. Finally, let (X, S) be an answer of W° that is not an MCP of W_t, i.e., $\mathcal{D}[S] = X$. By Lemma 6(2), $X \subseteq O_t$ and $(X, S \cup \{t\})$ is an MCP of W_t; it is an answer that contains t, so it is returned in line 13, and the replacement test removes the key S. The object sets of the remaining entries are not subsets of O_t, so by Lemma 6(2), these entries are still MCPs of W_t with unchanged frame and object sets, and hence still answers. Therefore $\mathcal{D} = Ans(W_t)$ after the arrival. □

Theorem 2 implies that both pruning rules are permanent: $\mathcal{D}$ never holds an entry with fewer than d frames or fewer than k objects, and the candidates discarded by the arrival DP are never stored either. The proof shows why this is safe. Every answer of W_t either is an entry of $\mathcal{D}$ that has lost the expiring frame, possibly merged with other entries, or contains t and is recomputed from the bitmaps. Neither source needs an entry outside Ans_{t-1}: by Lemma 5(1), an answer of W° comes from an answer of W_{t-1} with the same object set and at least as many frames, and an answer that contains t is computed without consulting $\mathcal{D}$. In other words, a stored entry never gains frames; it only loses the expiring frame or is replaced by a recomputed entry. An entry that fails the duration or the size threshold now can therefore never turn into an answer later; if its objects co-occur long enough in a later window, the arrival DP finds them again from their bitmaps.

For example, we run Algorithm 2 on the object stream of Fig. 1 with $w = 5$, starting at $t_0 = 4$, and trace the initialization and the slide caused by frame 5 (Fig. 6). We use $CQ(5, 2, 1)$, so that $(\{C\}, \{0, 2, 3\})$ is an answer of W_4 and every step is exercised. Since $t_0 < w$, positions coincide with frame numbers, and Algorithm 1 on the bitmaps of Fig. 2 initializes $\mathcal{D}$ to the four MCPs of W_4 with at least two frames, i.e., all but $(\{A, B, C, D\}, \{3\})$, registered in I as in Section IV-A.

When frame 5 arrives, frame 0 at position $p = 0$ expires. Since $O_0 = \{C\}$, bit 0 is cleared only in $F_4(C) = \texttt{10110}$, giving $F_{W^\circ}(C) = \texttt{00110}$, which is not empty and thus not recycled. $I[0]$ holds only $\texttt{10110} \mapsto \{C\}$, which is trimmed to $\texttt{00110}$; this key already holds the larger group $\{A, B, C\}$, so $\{C\}$ merges into it, because without frame 0, C appears only in frames 2 and 3, together with A and B. Next, bit 0, which now stands for frame 5, is set in the bitmaps of A, B, and D, giving $F_5(A) = F_5(B) = \texttt{11111}$ and $F_5(D) = \texttt{11011}$. Algorithm 1 on these three bitmaps returns $\texttt{11111} \mapsto \{A, B\}$ and $\texttt{11011} \mapsto \{A, B, D\}$, and each replaces the entry under its key without bit 0, $\texttt{01111}$ or $\texttt{01011}$, whose object set has the same size. The entry $\texttt{00110} \mapsto \{A, B, C\}$ contains neither frame 0 nor frame 5, so it stays untouched, and it remains an answer since C is absent from frame 5; the three entries now form Ans_5. For $CQ(5, 3, 3)$, the initialization leaves only $\texttt{01011} \mapsto \{A, B, D\}$, the expiration touches nothing, and the arrival replaces it with $\texttt{11011} \mapsto \{A, B, D\}$, which is Ans_5.

D. Complexity Analysis

a) Time per frame: The expiration clears bit p, which lies in a single machine word, in each of the $|O_{t-w}|$ bitmaps of the expiring frame and checks it for recycling, in $O(1)$ word operations thanks to the maintained numbers of frames; it then trims the $C_e = |I[pos(t)]|$ entries that contain frame $t-w$, with $O(\lceil w/64 \rceil)$ word operations each. The arrival sets $|O_t|$ bits and runs Algorithm 1 on O_t, intersecting each object with every entry of the working table $\mathcal{D}'$; with $|\mathcal{D}'|$ the largest size of $\mathcal{D}'$ during the run, this takes $O(|O_t| \cdot |\mathcal{D}'| \cdot \lceil w/64 \rceil)$ word operations, which also covers the replacement tests and insertions of the at most $|\mathcal{D}'|$ answers returned. In total, frame t takes $O\big(|O_{t-w}| + |O_t| + (C_e + |O_t| \cdot |\mathcal{D}'|) \cdot \lceil w/64 \rceil\big)$ word operations, and entries that contain neither the expiring nor the new frame cost nothing. Unlike a rerun of Algorithm 1 on the whole window, the arrival DP scans only the objects of the new frame instead of all n_t objects of W_t, and its working table holds only candidates that contain frame t.

b) Worst case: Every key of the arrival DP is $F_t(Y)$ for some $Y \subseteq O_t$, and every entry that contains frame $t - w$ has its object set in O_{t-w}; hence $|\mathcal{D}'| \leq 2^{|O_t|}$ and $C_e \leq 2^{|O_{t-w}|}$, and the cost of a frame is bounded by a function of the number of objects per frame, independently of the number of MCPs in the window and of the length of the stream. In practice, C_e and $|\mathcal{D}'|$ are far below these bounds and shrink as d grows, since Rule 1 discards objects and candidates with fewer than d frames and Rule 2 discards groups that cannot reach k objects. By contrast, the frame-wise methods of [1] maintain all MCPs of the window regardless of d and visit every maintained state in each frame, so their cost per frame grows with the number of MCPs, which can exceed $|Ans_t|$ by orders of magnitude.

c) Space: OWIM keeps one appearance bitmap for each of the n_t objects in W_t, the $\sum_{i \in W_t} |O_i|$ object identifiers of the frame ring, and the $|Ans_t|$ entries of $\mathcal{D}$ and I, each with a bitmap and an object set, in total $O\Big(n_t\lceil w/64 \rceil + \sum_{i \in W_t} |O_i| + \sum_{(X,S) \in Ans_t} \big(\lceil w/64 \rceil + |X|\big)\Big)$. Without recycling, a bitmap would be kept for every object seen since the start of the stream, and the space would grow without bound.

TABLE II
PARAMETER SETTINGS (DEFAULTS IN BOLD)

Param.	Meaning	Values
w	window size (frames)	**300**, 400, 500, 600, 900, 1200, 1800
d	duration threshold (frames)	1, 30, 60, 120, 180, 210, **240**, 270
k	size threshold (objects)	**1**, 2, 3, 5, 7, 9
p_o	max. reuses of an identifier	**0**, 1, 2, 3
p_r	fraction of removed objects	**0**, 0.1, 0.2, 0.3

V. EXPERIMENTS

We conduct all experiments on the same machine, with an Intel Core i7-13700F CPU, 32 GB of memory, and Windows 11. All baselines use the Java 8 implementation provided by the original authors of [1], without any modification. OWIM is also implemented in Java 8.

We report four metrics. 1) *total maintenance time*, i.e., the time to process the whole object stream. 2) *number of entries maintained and accessed per frame*. For the methods of [1], these are the number of states maintained in the window and the number of states accessed per frame; for OWIM, they are the number of entries in table $\mathcal{D}$, and the number of entries trimmed at expiration plus the number of intersections performed by Algorithm 1. Both counts directly reflect the work done by the algorithms. 3) *per-frame latency*, i.e., the time to process one frame. A continuous query must produce its answers promptly at every frame, so besides the mean we also report the 99th percentile, which reflects the slowest frames. 4) *memory*, i.e., the peak memory during maintenance.

A. Experimental Setup

Datasets. For a direct comparison with [1], our object streams are the detection and tracking results on the same videos, as provided by the authors of [1]. The datasets consist of six regular videos and three long videos. V1 and V2 are two synthetic videos generated by the Visual Road benchmark [13], showing rainy weather with light traffic and post-rain conditions with heavy traffic, respectively. D1 and D2 are MVI_40171 and MVI_40751 from UA-DETRAC [14], captured by static cameras. M1 and M2 are MOT16-06 and MOT16-13 from MOT16 [15], captured by moving cameras. The detection and tracking work used by the original authors are YOLOv3 [16] and DeepSORT [12]. The three long videos are publicly available videos on YouTube [17]–[19], with 36,000 to 54,000 frames each; their detection and tracking results are also provided by the original authors.

Baselines. We compare OWIM with three state maintenance methods from [1]: the naive method NAIVE, and the methods MFS and SSG proposed there. All three adopt Frame-Wise Incremental Maintenance, as described in Section II-B.

The paper [1] also provides a pruning technique for MFS and SSG: if the object set of a state fails the query, then

Fig. 7. Total maintenance time

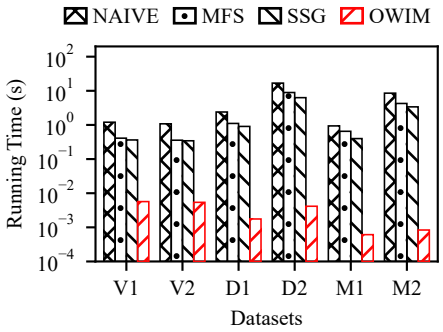


TABLE III
PER-FRAME STATES OF THE METHODS OF [1] (ALL ACCESSED) AND ENTRIES OF TABLE $\mathcal{D}$ MAINTAINED AND ACCESSED BY OWIM

	Methods of [1]		OWIM		
Dataset	NAIVE	MFS/SSG	maint.	acc.	Answers
V1	408.2	366.4	4.2	4.9	4.2
V2	325.9	289.5	3.8	5.5	3.8
D1	1348.2	1244.7	1.1	0.9	1.1
D2	6813.5	6623.2	2.3	3.2	2.3
M1	1770.8	1677.5	0.0	0.0	0.0
M2	7779.1	6882.0	0.7	0.6	0.7

TABLE IV
LONG VIDEOS: TOTAL MAINTENANCE TIME, AND STATES OF SSG OR ENTRIES OF TABLE $\mathcal{D}$ OF OWIM PER FRAME

		L1	L2	L3
Frames		51010	53405	35376
Time (s)	NAIVE	879.3	194.5	184.1
	MFS	427.2	71.7	69.4
	SSG	308.2	61.4	54.2
	OWIM	0.132	0.216	0.271
SSG/OWIM		2342×	284×	200×
Per frame	SSG	8226.4	1742.8	2492.5
	OWIM	4.6	7.0	10.3

every state derived from it has an object set contained in it and fails the query as well; such states are terminated and no longer processed. MFS and SSG with this pruning are denoted by MFS_O and SSG_O, respectively. When $k = 1$, every state satisfies the query, so the pruning has no effect and only adds the cost of evaluating the query on every new state. We therefore compare with MFS and SSG when $k = 1$, and with MFS_O and SSG_O when $k > 1$; NAIVE runs unchanged under all parameters. In this way, whenever the pruning of [1] can take effect, we compare against its pruned versions.

Parameter settings. Table II lists the parameters and their values, with the defaults underlined; in each experiment, we vary one parameter and set the others to their defaults, except in the experiment on k (see below). The defaults of the window size w and the duration threshold d are the same as in [1], namely $w = 300$ and $d = 240$; at 30 frames per second, this finds the objects that appear together for at least 8 seconds within a window of 10 seconds. The state maintenance experiments of [1] do not constrain the size of object sets, which corresponds to $k = 1$; we therefore also set the default of k to 1, so that the default setting is exactly that of [1]. The occlusion parameter p_o and the removal ratio p_r follow the definitions and values of [1]: p_o is the maximum number of times that the identifier of an object that has left the view is reused by later objects, which simulates more occlusions; p_r is the fraction of objects randomly removed from a video, which varies the number of objects in it. For these two parameters, we directly use the files generated by the original authors, which cover only the six regular videos.

Beyond the values used in [1], we extend three parameters. First, we extend d from 180–270 to 1–270. The methods of [1] maintain the same states for every d, so their time hardly changes with d, whereas OWIM keeps only the answers, so the larger d is, the fewer entries it keeps and processes; d thus has the largest impact on its cost, which the smaller values examine over the whole range. In particular, with $d = 1$ every MCP of the window is an answer, so all methods maintain the same MCPs, which shows the cost without pruning. Second, k is the size threshold of our query; we vary k from 1 to 9 and compare our permanent pruning by k with MFS_O and SSG_O. Here we set $d = 30$: with $d = 240$, no dataset has a group of five or more objects that co-occur in 240 frames, so the answer set would be empty for $k \geq 5$, leaving nothing to compare. Third, we extend w from 300–600 to 1800, i.e., 60 seconds: our bitmaps have w bits and each intersection takes $\lceil w/64 \rceil$ word operations, so longer windows make each operation more expensive.

B. Overall Performance

Six regular videos. Fig. 7 shows the total maintenance time on the six regular videos under the default parameters ($w = 300$, $d = 240$, $k = 1$). OWIM takes only 0.6–5.7 ms, versus 0.34–6.30 s for SSG, 0.36–8.91 s for MFS, and 0.94–16.81 s for NAIVE; it is 64–4,055 times faster than SSG and 200–10,219 times faster than NAIVE. As in [1], SSG is the fastest baseline: it is 2.4–3.3 times faster than NAIVE and takes 4%–39% less time than MFS.

Table III explains the gap. Since the methods of [1] do not prune by d, at every frame they maintain and access all MCPs of the window: 290–6,882 states for MFS and SSG, and 326–7,779 for NAIVE, which also keeps object sets that are not MCPs. Yet there are only 0–4.2 answers per frame on average; since its table is exactly the answer set, OWIM maintains only these entries and accesses only 0–5.5 entries per frame. The gap is thus determined by the ratio of the number of MCPs to the number of answers. It is the largest on D2 and M2, whose windows contain 6,623 and 6,882 MCPs on average, where OWIM is 1,518 and 4,055 times faster than SSG, and the smallest on V1 and V2, where objects stay long and rarely enter or leave the view, so the windows contain only 290–366 MCPs; even there, OWIM is 64 times faster. On M1, captured by a moving camera with the shortest object durations, no group of objects co-occurs in 240 frames, so the answer set and the table of OWIM stay empty: each frame only updates the bitmaps of the objects in the new and the expiring frames, and Rule 1 discards every object of the new frame, as each appears in fewer than 240 frames, while the methods of [1] still maintain 1,678 states per frame.

Long videos. On the three long videos of 35,000 to 53,000 frames, under the default parameters (Table IV), the methods of [1] take 54–879 s and OWIM only 0.13–0.27 s, i.e., 200–2,342 times faster than SSG and 680–6,682 times faster than

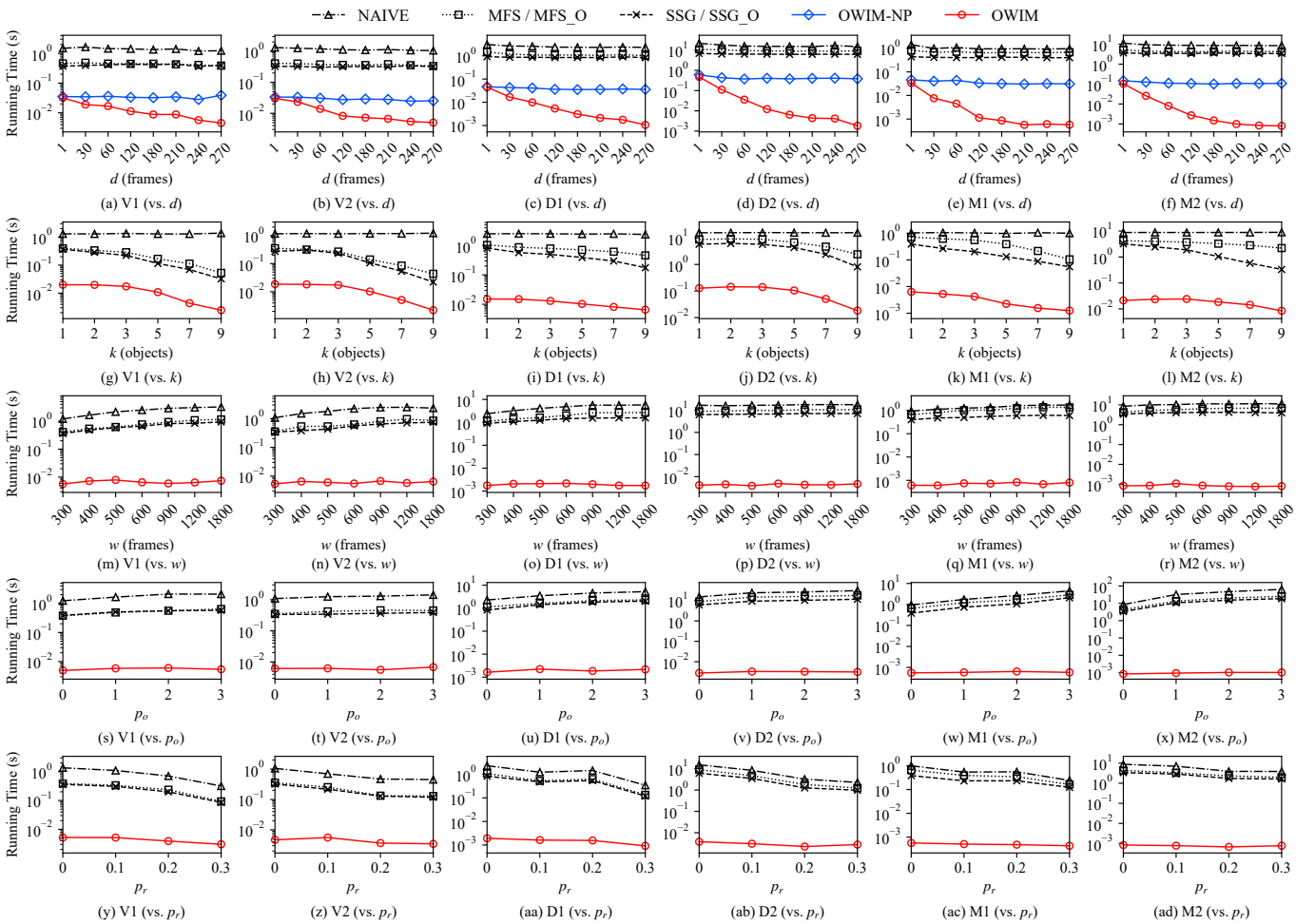


Fig. 8. Total maintenance time when varying d, k, w, p_o, and p_r: (a)–(f) varying d; (g)–(l) varying k with $d = 30$, where MFS and SSG are used when $k = 1$ and MFS_O and SSG_O when $k > 1$; (m)–(r) varying w; (s)–(x) varying p_o; (y)–(ad) varying p_r

NAIVE. The baselines again agree with [1]: SSG and MFS are 2.9–3.4 and 2.1–2.7 times faster than NAIVE, versus 3.2 and 2.6 times reported there for a long video. The gap is the largest on L1, which has about twice as many objects per frame as the other two: SSG maintains 8,226 states per frame and takes 308 s, whereas OWIM holds only 4.6 entries on average and its total time on L1 is the shortest of the three videos, so its cost depends on the number of answers rather than on the number of objects or MCPs. Since its table holds only the current answers and the bitmaps of objects that have left the window are recycled, the data maintained by OWIM does not grow with the length of the stream.

C. Effect of Query Parameters

Figs. 8(a)–(r) show the total maintenance time of all methods on the six videos as d, k, and w vary.

Varying d. In Figs. 8(a)–(f), the methods of [1] hardly change with d, since they maintain all MCPs of the window regardless of d: on each dataset, their longest and shortest times differ by a factor of only 1.1–1.5. OWIM, in contrast, becomes rapidly faster as d grows, by a factor of 6–261 from $d = 1$ to $d = 270$. To isolate the effect of permanent pruning, we add OWIM-NP, which maintains table $\mathcal{D}$ with $d = 1$ and $k = 1$ and filters the answers by d and k only when reporting them; like the methods of [1], it maintains all MCPs, and its curve hardly changes with d. With $d = 1$, nothing is pruned and the two OWIM curves almost coincide, OWIM-NP taking 4%–41% more time to scan the whole table at every frame for the answers. Even then, OWIM is 11–35 times faster than SSG: both maintain the same MCPs, but SSG accesses all of its 290–6,882 states at every frame, whereas OWIM accesses only the entries that contain the new or the expiring frame, 2.7–13 times fewer, and each intersection is a bitwise AND of bitmaps. As d grows, the two OWIM curves quickly separate: at the default $d = 240$, SSG is 13–32 times slower than OWIM-NP, a gap unrelated to pruning, and OWIM-NP is 5–127 times slower than OWIM, a gap due to permanent pruning.

Varying k. Figs. 8(g)–(l) vary k with $d = 30$, using MFS and SSG for $k = 1$, where they have nothing to prune, and their pruned versions MFS_O and SSG_O for $k > 1$. NAIVE does not prune by k and stays unchanged, whereas the other three methods become faster as k grows. On most datasets,

the answers change little for $k \leq 3$ (e.g., on V1 they drop by only 15% from $k = 1$ to $k = 3$), and so does the time of OWIM; for $k \geq 5$, the answers decrease quickly, and OWIM becomes faster by a factor of 2.3–8.4 from $k = 1$ to $k = 9$. Over all values of k, OWIM is 10–152, 14–261, and 62–1,060 times faster than SSG_O, MFS_O, and NAIVE, respectively. SSG_O usually drops the fastest (by a factor of 4.7–12.3), and on M2 the advantage of OWIM over it shrinks from 152 to 39 times; however, SSG_O misses more answers as k grows (96% on M2 with $k = 9$), so its time is biased low.

Varying w. In Figs. 8(m)–(r), with $d = 240$, the methods of [1] slow down by 6%–181% from $w = 300$ to $w = 1800$, since a longer window contains more MCPs (e.g., the states per frame grow from 1,245 to 3,923 on D1 and from 6,882 to 13,962 on M2). The time of OWIM stays almost unchanged, within a factor of 1.4 on each dataset: the answers per frame grow with w (e.g., from 4.2 to 24.4 on V1) but remain at most a few dozen, and the bitmaps grow from 5 to 29 words, which makes each intersection more expensive but hardly affects the total time. With $w = 1800$, OWIM is 117–4,954 times faster than SSG.

D. Effect of Data Characteristics

Figs. 8(s)–(ad) show the total maintenance time of all methods on the six videos as p_o and p_r vary, with the other parameters at their defaults.

Varying p_o. In Figs. 8(s)–(x), the methods of [1] slow down by a factor of 1.2–7.5 from $p_o = 0$ to $p_o = 3$, most on M1 and M2, which are captured by moving cameras (by a factor of 5.4 and 5.6 for SSG). A reused identifier stands for several objects that appear one after another, so its frames in the window are scattered and it co-occurs with more objects; the window thus contains more MCPs, e.g., the states per frame on M2 grow from 6,839 to 32,766. The answers that satisfy the duration threshold, however, hardly change (0–4.8 per frame), so the time of OWIM grows by at most 32%; the more occlusions, the larger its advantage, and with $p_o = 3$ it is 57–18,371 times faster than SSG.

Varying p_r. In Figs. 8(y)–(ad), removing objects reduces both the objects per frame and the MCPs in the window, so all methods become faster. The methods of [1] benefit the most, their time dropping to 12%–48% of the original from $p_r = 0$ to $p_r = 0.3$ (e.g., the states per frame on D2 drop from 6,623 to 1,196), whereas the time of OWIM, which maintains only the answers, drops only to 48%–92%. The advantage of OWIM thus shrinks, yet with $p_r = 0.3$ it is still 28–2,104 times faster than SSG. Since the removed objects are drawn independently for each value of p_r, the curves are not strictly monotonic; e.g., on D1 the methods of [1] are slightly slower with $p_r = 0.2$ than with $p_r = 0.1$.

E. Per-frame Latency and Memory

Per-frame latency. Table V reports the mean, the 99th percentile, and the maximum of the per-frame latency of SSG and OWIM under the default parameters; MFS and NAIVE are slower than SSG and are omitted. OWIM takes 0.4–7.7 μs per frame on average, at most 53 μs at the 99th percentile, and less than 0.5 ms in the worst case, whereas SSG takes 0.19–6.0 ms, 0.56–49.5 ms, and up to 121 ms, respectively. At 30 frames per second, a frame must be processed within about 33 ms; on D2, state maintenance alone exceeds this budget on 103 frames (9%) for SSG and on 111 and 120 frames for MFS and NAIVE, whereas the slowest frame of OWIM there takes only 33 μs.

TABLE V
PER-FRAME LATENCY (μS) UNDER THE DEFAULT PARAMETERS

	SSG			OWIM		
Dataset	mean	p99	max	mean	p99	max
V1	198	557	623	3.0	10.8	13.9
V2	194	623	854	3.2	12.2	14.3
D1	767	2,609	3,236	1.4	4.4	12.6
D2	5,449	49,550	51,805	3.0	13.0	33.4
M1	334	1,053	1,220	0.4	1.0	4.4
M2	4,628	13,780	15,806	1.1	3.6	19.3
L1	6,042	29,254	121,251	2.6	38.6	482.3
L2	1,150	5,571	22,220	4.1	21.2	431.6
L3	1,532	9,380	26,099	7.7	52.7	499.6

Fig. 9 shows the latency of every frame on D2 and M2. The latency of SSG rises and falls with the number of states it maintains, with a correlation coefficient of 0.95–1.00. In the later part of D2, more objects are in view, and the states maintained by SSG reach 57,726 at frame 1,114, which takes SSG 52 ms to process. The latency of OWIM stays within 0.2–33 μs on both videos; even in the last part of D2, where SSG is the slowest, OWIM takes at most 15 μs per frame.

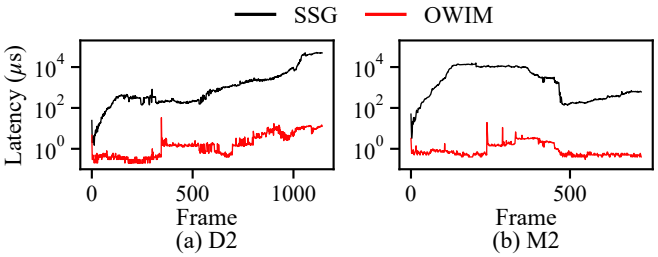


Fig. 9. Per-frame latency on D2 and M2 under the default parameters

Memory. Table VI reports the peak memory of each method during maintenance under the default parameters. To measure it, we periodically run a full garbage collection during maintenance and record the live heap of the JVM in excess of that after the input is loaded. OWIM takes only 39–57 KB on all datasets: its table holds only the answers, appearance bitmaps are kept only for the objects of the current window, and the bitmaps of objects that leave the window are recycled. The methods of [1] keep all MCPs of the window, so their memory grows with the number of states; SSG takes 1.06–68.07 MB, 22–1,235 times as much as OWIM. OWIM-NP maintains the same MCPs as SSG but takes 2.8–5.5 times less memory, since it represents a frame set by a w-bit bitmap, whereas the

TABLE VI
PEAK MEMORY (MB) UNDER THE DEFAULT PARAMETERS

Dataset	NAIVE	MFS	SSG	OWIM-NP	OWIM
V1	0.664	1.036	1.064	0.280	0.048
V2	0.967	1.644	1.685	0.376	0.047
D1	2.063	3.362	3.550	0.845	0.045
D2	40.982	64.998	68.065	12.346	0.055
M1	1.361	1.973	2.124	0.758	0.038
M2	8.545	14.276	14.952	3.239	0.049

implementation of [1] keeps a per-frame list of frame numbers for each state.

VI. Related Work

Query processing over videos. Much work on declarative video analytics reduces the cost of deep models with cheap filters and specialized models [20], [21], probabilistic predicates [22], and query optimization [23]–[25]; other work supports richer predicates over the detected objects, such as spatial constraints [26], object tracks [27], compositional events [28], and complex event patterns over video streams [29]. These techniques are orthogonal to ours, which works on the object stream produced by detection and tracking. Closest to our work, Chen et al. [1] introduce co-occurrence queries over video feeds with two maintenance methods, MFS and SSG. A follow-up [2] returns the top-ranked windows of stored videos by the same co-occurrence score; its offline indexes over video partitions and its early termination do not apply to our continuous queries, which must report all answers at every frame as the frames arrive. Other follow-ups retrieve clips of stored videos under spatial and temporal constraints [3] and track groups of spatially close objects frame by frame [11]; both require spatial relationships among objects, whereas co-occurrence queries only require the objects to appear in the same frame.

Co-movement patterns. Co-occurrence queries resemble co-movement patterns, i.e., groups of objects that stay spatially close for a minimum duration, where proximity is determined either by clustering object locations at each time point for GPS trajectories, e.g., with DBSCAN [30]–[32], or by close entrance times at the same camera for trajectories across a road network [4], [5]. A closed swarm [30], whose time points may be non-consecutive, is an MCP when all objects of every frame form one cluster; its miner, ObjectGrowth, searches object by object and prunes by duration, like our computation for a single window, but mines a whole database offline. Later work mines swarms in sliding windows [31], detects co-movement patterns on streaming trajectories with distributed processing [32], and mines offline the groups that travel together along a route of cameras [4], [5]. In contrast, co-occurrence queries work on the object stream of a single camera and only require objects to appear in the same frame, with neither object locations nor clustering.

Object detection and tracking. Our object streams come from object detection and multi-object tracking, whose advances are orthogonal to this work: detectors have evolved from two-stage designs [33] to one-stage real-time [6], [16], [34] and transformer-based ones [7], [35], and most trackers follow tracking by detection, associating detections across frames by motion and appearance [8]–[10], [12], [36]. Occlusions and identity switches still fragment tracks; co-occurrence queries tolerate short occlusions, as an object set needs to co-occur in only d of the w frames, and fragmented tracks can be merged during video ingestion [37].

VII. Conclusion

We studied continuous co-occurrence queries over object streams extracted from video feeds and proposed OWIM, an incremental maintenance method that scans object by object: candidates are pruned by both the duration and the size thresholds, the pruning remains safe as the window slides, and the maintained table is exactly the answer set at every frame. With circularly encoded appearance bitmaps and an index on the oldest frame of each entry, each frame processes only the entries that contain the expiring or the new frame. On the same datasets as [1], OWIM is 64–4,055 times faster than SSG under the default parameters and 200–2,342 times faster on long videos, with a 99th-percentile latency of at most 53 μs per frame.

References

[1] Y. Chen, X. Yu, N. Koudas, and Z. Yu, "Evaluating temporal queries over video feeds," in *SIGMOD '21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021*, G. Li, Z. Li, S. Idreos, and D. Srivastava, Eds. ACM, 2021, pp. 287–299. [Online]. Available: https://doi.org/10.1145/3448016.3452803

[2] Y. Chen, X. Yu, and N. Koudas, "Ranked window query retrieval over video repositories," in *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9-12, 2022*. IEEE, 2022, pp. 2776–2791. [Online]. Available: https://doi.org/10.1109/ICDE53745.2022.00253

[3] Y. Chen, N. Koudas, X. Yu, and Z. Yu, "Spatial and temporal constrained ranked retrieval over videos," *Proc. VLDB Endow.*, vol. 15, no. 11, pp. 3226–3239, 2022. [Online]. Available: https://www.vldb.org/pvldb/vol15/p3226-yu.pdf

[4] D. Zhang, T. Ma, J. Hu, Y. Bei, K.-L. Tan, and G. Chen, "Co-movement pattern mining from videos," *Proc. VLDB Endow.*, vol. 17, no. 3, pp. 604–616, 2023. [Online]. Available: https://www.vldb.org/pvldb/vol17/p604-zhang.pdf

[5] Y. Bei, T. Ma, D. Zhang, S. Wu, K.-L. Tan, and G. Chen, "Mining platoon patterns from traffic videos," *Proc. VLDB Endow.*, vol. 18, no. 6, pp. 1839–1851, 2025. [Online]. Available: https://www.vldb.org/pvldb/vol18/p1839-bei.pdf

[6] Y. Tian, Q. Ye, and D. S. Doermann, "YOLOv12: Attention-centric real-time object detectors," in *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025, San Diego, CA, USA, December 2-7, 2025 / Mexico City, Mexico, November 30 - December 5, 2025*, D. Belgrave, C. Zhang, L. N. Montoya, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, N. Chen, I. V. M. Ruíz, and A. Loaiza-Bonilla, Eds., 2025.

[7] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "DETRs beat YOLOs on real-time object detection," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024, Seattle, WA, USA, June 16-22, 2024*. IEEE, 2024, pp. 16 965–16 974. [Online]. Available: https://doi.org/10.1109/CVPR52733.2024.01605

[8] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in *Computer Vision - ECCV 2022 - 17th European Conference, Tel Aviv, Israel, October 23-27, 2022, Proceedings, Part XXII*, ser. Lecture Notes in Computer Science, S. Avidan, G. J. Brostow, M. Cissé, G. M. Farinella, and T. Hassner, Eds., vol. 13682. Springer, 2022, pp. 1–21. [Online]. Available: https://doi.org/10.1007/978-3-031-20047-2_1

[9] J. Cao, J. Pang, X. Weng, R. Khirodkar, and K. Kitani, "Observation-centric SORT: rethinking SORT for robust multi-object tracking," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023, Vancouver, BC, Canada, June 17-24, 2023*. IEEE, 2023, pp. 9686–9696. [Online]. Available: https://doi.org/10.1109/CVPR52729.2023.00934

[10] Y. Du, Z. Zhao, Y. Song, Y. Zhao, F. Su, T. Gong, and H. Meng, "StrongSORT: Make DeepSORT great again," *IEEE Trans. Multim.*, vol. 25, pp. 8725–8737, 2023. [Online]. Available: https://doi.org/10.1109/TMM.2023.3240881

[11] H. Yoon, D. Choi, and Y. D. Chung, "Group tracking for video monitoring systems: A spatio-temporal query processing approach," *IEEE Access*, vol. 11, pp. 19 969–19 987, 2023. [Online]. Available: https://doi.org/10.1109/ACCESS.2023.3249190

[12] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in *2017 IEEE International Conference on Image Processing, ICIP 2017, Beijing, China, September 17-20, 2017*. IEEE, 2017, pp. 3645–3649. [Online]. Available: https://doi.org/10.1109/ICIP.2017.8296962

[13] B. Haynes, A. Mazumdar, M. Balazinska, L. Ceze, and A. Cheung, "Visual Road: A video data management benchmark," in *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019*. ACM, 2019, pp. 972–987. [Online]. Available: https://doi.org/10.1145/3299869.3324955

[14] S. Lyu, M. Chang, D. Du, L. Wen, H. Qi, Y. Li, Y. Wei, L. Ke, T. Hu, M. Del Coco *et al.*, "UA-DETRAC 2017: Report of AVSS2017 & IWT4S challenge on advanced traffic monitoring," in *14th IEEE International Conference on Advanced Video and Signal Based Surveillance, AVSS 2017, Lecce, Italy, August 29 - September 1, 2017*. IEEE, 2017, pp. 1–7. [Online]. Available: https://doi.org/10.1109/AVSS.2017.8078560

[15] A. Milan, L. Leal-Taixé, I. D. Reid, S. Roth, and K. Schindler, "MOT16: A benchmark for multi-object tracking," *CoRR*, vol. abs/1603.00831, 2016. [Online]. Available: http://arxiv.org/abs/1603.00831

[16] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," *CoRR*, vol. abs/1804.02767, 2018. [Online]. Available: http://arxiv.org/abs/1804.02767

[17] "YouTube video," https://www.youtube.com/watch?v=wqctLW0Hb_0.

[18] "YouTube video," https://www.youtube.com/watch?v=e_WBuBqS9h8.

[19] "YouTube video," https://www.youtube.com/watch?v=Inbd-YOdPUE.

[20] D. Kang, J. Emmons, F. Abuzaid, P. Bailis, and M. Zaharia, "NoScope: Optimizing neural network queries over video at scale," *Proc. VLDB Endow.*, vol. 10, no. 11, pp. 1586–1597, 2017. [Online]. Available: http://www.vldb.org/pvldb/vol10/p1586-kang.pdf

[21] D. Kang, P. Bailis, and M. Zaharia, "BlazeIt: Optimizing declarative aggregation and limit queries for neural network-based video analytics," *Proc. VLDB Endow.*, vol. 13, no. 4, pp. 533–546, 2019. [Online]. Available: http://www.vldb.org/pvldb/vol13/p533-kang.pdf

[22] Y. Lu, A. Chowdhery, S. Kandula, and S. Chaudhuri, "Accelerating machine learning inference with probabilistic predicates," in *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018, Houston, TX, USA, June 10-15, 2018*, G. Das, C. M. Jermaine, and P. A. Bernstein, Eds. ACM, 2018, pp. 1493–1508. [Online]. Available: https://doi.org/10.1145/3183713.3183751

[23] J. Cao, K. Sarkar, R. Hadidi, J. Arulraj, and H. Kim, "FiGO: Fine-grained query optimization in video analytics," in *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*, Z. G. Ives, A. Bonifati, and A. E. Abbadi, Eds. ACM, 2022, pp. 559–572. [Online]. Available: https://doi.org/10.1145/3514221.3517857

[24] J. Bang, G. T. Kakkar, P. Chunduri, S. Mitra, and J. Arulraj, "SEIDEN: revisiting query processing in video database systems," *Proc. VLDB Endow.*, vol. 16, no. 9, pp. 2289–2301, 2023. [Online]. Available: https://www.vldb.org/pvldb/vol16/p2289-kakkar.pdf

[25] D. Chao, Y. Chen, N. Koudas, and X. Yu, "Optimizing video queries with declarative clues," *Proc. VLDB Endow.*, vol. 17, no. 11, pp. 3256–3268, 2024. [Online]. Available: https://www.vldb.org/pvldb/vol17/p3256-koudas.pdf

[26] N. Koudas, R. Li, and I. Xarchakos, "Video monitoring queries," in *36th IEEE International Conference on Data Engineering, ICDE 2020, Dallas, TX, USA, April 20-24, 2020*. IEEE, 2020, pp. 1285–1296. [Online]. Available: https://doi.org/10.1109/ICDE48307.2020.00115

[27] F. Bastani, S. He, A. Balasingam, K. Gopalakrishnan, M. Alizadeh, H. Balakrishnan, M. J. Cafarella, T. Kraska, and S. Madden, "MIRIS: fast object track queries in video," in *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020, online conference [Portland, OR, USA], June 14-19, 2020*, D. Maier, R. Pottinger, A. Doan, W. Tan, A. Alawini, and H. Q. Ngo, Eds. ACM, 2020, pp. 1907–1921. [Online]. Available: https://doi.org/10.1145/3318464.3389692

[28] E. Zhang, M. Daum, D. He, B. Haynes, R. Krishna, and M. Balazinska, "EQUI-VOCAL: synthesizing queries for compositional video events from limited user interactions," *Proc. VLDB Endow.*, vol. 16, no. 11, pp. 2714–2727, 2023. [Online]. Available: https://www.vldb.org/pvldb/vol16/p2714-zhang.pdf

[29] C. Han, C. Chang, S. Srivastava, Y. Lu, and E. Lo, "Scalable complex event processing on video streams," *Proc. ACM Manag. Data*, vol. 3, no. 3, pp. 214:1–214:29, 2025. [Online]. Available: https://doi.org/10.1145/3725419

[30] Z. Li, B. Ding, J. Han, and R. Kays, "Swarm: Mining relaxed temporal moving object clusters," *Proc. VLDB Endow.*, vol. 3, no. 1, pp. 723–734, 2010. [Online]. Available: https://vldb.org/pvldb/vol3/R65.pdf

[31] A. Bhushan, U. Bellur, K. Sharma, S. Deshpande, and N. L. Sarda, "Mining swarm patterns in sliding windows over moving object data streams," in *Proceedings of the 25th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems, GIS 2017, Redondo Beach, CA, USA, November 7-10, 2017*, E. G. Hoel, S. D. Newsam, S. Ravada, R. Tamassia, and G. Trajcevski, Eds. ACM, 2017, pp. 60:1–60:4. [Online]. Available: https://doi.org/10.1145/3139958.3139988

[32] L. Chen, Y. Gao, Z. Fang, X. Miao, C. S. Jensen, and C. Guo, "Real-time distributed co-movement pattern detection on streaming trajectories," *Proc. VLDB Endow.*, vol. 12, no. 10, pp. 1208–1220, 2019. [Online]. Available: http://www.vldb.org/pvldb/vol12/p1208-chen.pdf

[33] S. Ren, K. He, R. B. Girshick, and J. Sun, "Faster R-CNN: towards real-time object detection with region proposal networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, 2017. [Online]. Available: https://doi.org/10.1109/TPAMI.2016.2577031

[34] J. Redmon, S. K. Divvala, R. B. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 2016, pp. 779–788. [Online]. Available: https://doi.org/10.1109/CVPR.2016.91

[35] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23-28, 2020, Proceedings, Part I*, ser. Lecture Notes in Computer Science, A. Vedaldi, H. Bischof, T. Brox, and J. Frahm, Eds., vol. 12346. Springer, 2020, pp. 213–229. [Online]. Available: https://doi.org/10.1007/978-3-030-58452-8_13

[36] A. Bewley, Z. Ge, L. Ott, F. T. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *2016 IEEE International Conference on Image Processing, ICIP 2016, Phoenix, AZ, USA, September 25-28, 2016*. IEEE, 2016, pp. 3464–3468. [Online]. Available: https://doi.org/10.1109/ICIP.2016.7533003

[37] D. Chao, Y. Chen, N. Koudas, and X. Yu, "Track merging for effective video query processing," in *39th IEEE International Conference on Data Engineering, ICDE 2023, Anaheim, CA, USA, April 3-7, 2023*. IEEE, 2023, pp. 164–176. [Online]. Available: https://doi.org/10.1109/ICDE55515.2023.00020